

Splitwise: Efficient Generative LLM Inference Using Phase Splitting

Pratyush Patel¹, Esha Choukse², Chaojie Zhang²,
Aashaka Shah², Íñigo Goiri², Saeed Maleki², Ricardo Bianchini²

¹University of Washington

²Microsoft

Abstract—Generative large language model (LLM) applications are growing rapidly, leading to large-scale deployments of expensive and power-hungry GPUs. Our characterization of LLM inference shows that each inference request undergoes two phases: a compute-intensive prompt computation phase and a memory-intensive token generation phase, each with distinct latency, throughput, memory, and power characteristics. Despite state-of-the-art batching and scheduling, the token generation phase underutilizes compute resources. Unlike prompt computation, token generation does not need the compute capability of the latest GPUs and can be run with lower power and cost.

Based on these insights, we propose Splitwise, a model deployment and scheduling technique that splits the two phases of LLM inference requests on to separate machines. Splitwise enables phase-specific resource management using hardware that is well suited for each phase. Request state is transferred efficiently between machines using optimized network libraries on the fast back-plane interconnects available in today’s GPU clusters. Using Splitwise, we design homogeneous and heterogeneous LLM inference clusters optimized for throughput, cost, and power. Compared to current designs, Splitwise clusters achieve up to 1.4× higher throughput at 20% lower cost. Alternatively, they can deliver 2.35× more throughput under the same power and cost budgets.

I. INTRODUCTION

Recent advancements in generative large language models (LLMs) have significantly improved their response quality and accuracy [18], [71]. These trends have led to the widespread adoption of LLMs across various domains [6], [21]. Most modern LLMs are built using the transformer architecture [77], [78] and exhibit similar characteristics [63]. Transformer model sizes have grown steadily, from the early BERT models [36] having 340 million parameters, to GPT-3 [28] with a staggering 175 billion parameters, and GPT-4 rumored to have even more.

LLMs typically run on expensive and power-hungry GPUs [16]. The sudden and large-scale deployment of LLMs has led to a worldwide GPU capacity crunch [14]. The computational demand for LLM inference far exceeds that of training due to the vast number of applications leveraging LLMs. Furthermore, since training LLMs requires expensive and dedicated supercomputers [56], [60], a large number of inferences are necessary to amortize the high training costs. LLM inference jobs, although orders of magnitude smaller than training, are still expensive given the compute involved.

	A100	H100	Ratio
TFLOPs	19.5	66.9	3.43×
HBM capacity	80GB	80GB	1.00×
HBM bandwidth	2039GBps	3352GBps	1.64×
Power	400W	700W	1.75×
NVLink	50Gbps	100Gbps	2.00×
Infiniband	200GBps	400GBps	2.00×
Cost per machine [5]	\$17.6/hr	\$38/hr	2.16×

TABLE I: NVIDIA A100 vs. H100 specifications.

Generative LLM inference for a single request consists of several forward passes through the model, since the output tokens are generated one by one. This inherently has two contrasting phases of computation. First, the *prompt computation phase*, in which all the input prompt tokens run through the forward pass of the model in parallel to generate the first output token. This phase tends to be computationally intensive and requires the high FLOPs (floating point operations per second) of the latest GPUs today. Second, the *token generation phase*, in which subsequent output tokens are generated sequentially based on the forward pass of the last token and all the cached context from previous tokens in the sequence. Given the lack of compute parallelism, this phase tends to be more memory bandwidth and capacity bound, despite state-of-the-art batching. Running both phases on the same machine often leads to inconsistent end-to-end latencies due to the arbitrary batching of prompt and token phases. Due to these challenges, services need to over-provision expensive GPUs to meet tight inference service level objectives (SLOs) for interactive applications. At the same time, cloud service providers (CSPs) are having to build a lot of new datacenters to meet the GPU demand, and are running into a power wall [19].

The industry continues to release new computationally powerful GPUs, each much more power hungry and expensive than the last. However, as shown in Table I, the high-bandwidth memory (HBM) capacity and bandwidth on these GPUs has not scaled at the same rate recently. The latest NVIDIA H100 GPUs have 3.43× more compute and 1.75× more power compared to their predecessor A100 GPUs. However, their memory bandwidth only grew by 1.6×, with no increase in memory capacity.

Our work. Given the distinct properties of prompt computation and token generation phases, we propose splitting the inference

¹Work partly done as an intern at Microsoft.

4
2
0
2
y
a
M
0
2
]
R
A
.
s
c
[
2
v
7
7
6
8
1
:
1
1
3
2
:
v
i
X
r
a

2 微软

表 I: NVIDIA A100 与 H100 规格。

由于输出标记是一一生成的，因此单个请求的生成 LLM 推理由模型中的多次前向传递组成。这本质上有两个对比的计算阶段。首先是 *prompt computation phase*，其中所有输入提示标记并行运行模型的前向传递以生成第一个输出标记。此阶段的计算量往往较大，并且需要当今最新 GPU 的高 FLOP（每秒浮点运算）。其次，*token generation phase*，其中基于最后一个令牌的前向传递以及序列中先前令牌的所有缓存上下文，顺序生成后续输出令牌。由于缺乏计算并行性，尽管采用了最先进的批处理技术，但此阶段往往会受到更多内存带宽和容量的限制。由于提示和令牌阶段的任意批处理，在同一台机器上运行这两个阶段通常会导致端到端延迟不一致。由于这些挑战，服务需要过度配置昂贵的 GPU，以满足交互式应用程序严格的推理服务级别目标 (SLO)。与此同时，云服务提供商 (CSP) 必须构建大量新的数据中心来满足 GPU 需求，并且遇到了电力墙 [19]。

生成大语言模型（LLM）的最新进展显着提高了其响应质量和准确性[18], [71]。这些趋势导致了大语言模型在多个领域的广泛采用[6]、[21]。大多数现代大语言模型使用变压器架构[77]、[78]构建的, 并表现出类似的特征[63]。Transformer 模型的大小一直在稳步增长, 从早期的 BERT 模型 [36] 拥有 3.4 亿个参数, 到 GPT-3 [28] 拥有惊人的 1750 亿个参数, 而据传 GPT-4 的参数甚至更多。

LLM 通常在昂贵且耗电的 GPU 上运行 [16]。大语言模型的突然大规模部署导致了全球范围内的 GPU 容量紧缩[14]。由于大量应用程序利用了 LLM，因此 LLM 推理的计算需求远远超过了训练的计算需求。此外，由于训练法学硕士需要昂贵的专用超级计算机[56]、[60]，因此需要大量的推理来分摊高昂的培训成本。LLM 推理作业虽然比训练小几个数量级，但考虑到所涉及的计算，仍然很昂贵。

业界不断发布新的计算能力强大的 GPU，每种 GPU 都比上一个更加耗电且昂贵。然而，如表一所示，这些 GPU 上的高带宽内存 (HBM) 容量和带宽最近并未以相同的速率扩展。与前代 A100 GPU 相比，最新的 NVIDIA H100 GPU 的计算能力提高了 3.43 $\times$ ，功耗提高了 1.75 $\times$ 。然而，它们的内存带宽仅增长了 1.6 $\times$ ，而内存容量没有增加。

我们的工作。考虑到提示计算和令牌生成阶段的不同属性，我们建议分割推理

¹Work partly done as an intern at Microsoft.

request and running them on separate machines. Doing so allows us to separately manage hardware resources for each phase, thereby increasing the GPU utilization and the overall efficiency of the system. It also enables using different, better-suited hardware for each phase. To realize such a setup, the cached context from the prompt computation needs to be communicated over from the prompt processing machine to the token generation machine at low latency. We implement these transfers in an optimized manner over the back-end Infiniband interconnects available in datacenters today, allowing us to increase efficiency without any perceived performance loss.

With Splitwise, we design clusters optimized for cost, throughput, and power, using production traces of LLM inference requests [4]. Given the diverging memory and compute scaling rates across GPU generations, we also evaluate different GPUs and power caps for the different inference phases. This allows us to target better performance per dollar (Perf/\$) for users, and better performance per watt (Perf/W) for CSPs. Additionally, users can target older GPUs, which are likely more readily available to them.

We show that Splitwise-based LLM inference clusters can achieve $1.4\times$ higher throughput at 20% lower cost than existing clusters. Alternatively, they can deliver $2.35\times$ more throughput with the same cost and power budgets.

Summary. We make the following contributions:

- 1) An extensive characterization of the differences in the execution and utilization patterns of the prompt and token generation phases in LLM inference on the NVIDIA A100 and H100 GPUs using production traces.
- 2) Splitwise, our technique for optimized utilization of available hardware, which splits the prompt computation and token generation phases onto separate machines.
- 3) A design exploration of homogeneous and heterogeneous cluster deployments with Splitwise to optimize the overall cost, request throughput, and provisioned power.
- 4) An evaluation of the systems designed with Splitwise using production traces.

II. BACKGROUND

A. Large Language Models

Modern LLMs are based on transformers. Transformer models use attention [77] and multi-layer-perceptron layers to understand the inputs and generate an output, respectively. Transformer-based LLMs include encoder-only [36], [54], decoder-only [67], [69], [71], and encoder-decoder [70] models. Generative LLMs, the focus of this paper, are usually either decoder-only, or encoder-decoder models.

B. Generative LLM inference phases

Figure 1 shows an example of generative LLM inference. Once the prompt query is received, all the input tokens are computed in parallel, within a single iteration, to generate the first token. We call this the prompt processing phase. The context generated from the attention layers during the prompt computation is saved in the key-value (KV) cache, since it

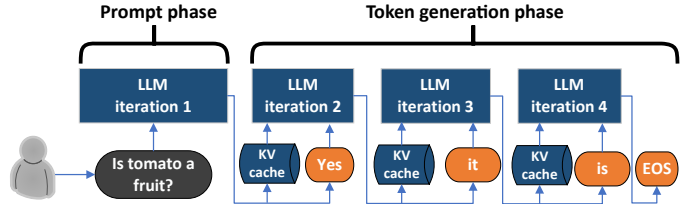


Fig. 1: An LLM inference example.

Metric	Importance to user
End-to-end (E2E) latency	Total query time that the user sees
Time to first token (TTFT)	How quickly user sees initial response
Time between tokens (TBT)	Average token streaming latency
Throughput	Requests per second

TABLE II: Performance metrics for LLMs.

is needed for all the future token generation iterations. After the first token is generated, the following tokens only use the last generated token and the KV-cache as inputs to the forward pass of the model. This makes the subsequent token generation more memory bandwidth and capacity intensive than the computationally heavy prompt phase.

C. Performance metrics for LLMs

Prior work has proposed three main metrics for LLM inference: end-to-end (E2E) latency, time to first token (TTFT), and throughput. We add another latency metric: time between tokens (TBT), to track the online streaming throughput of the tokens as they are generated serially. Table II summarizes the key performance metrics that we consider in this work.

Generative LLMs may be used for a variety of tasks with different kinds of SLOs. For batch tasks (e.g., summarization), TTFT or TBT latency metrics are less important than throughput. On the other hand, for latency-sensitive tasks (e.g., conversational APIs), TTFT and TBT are the more important metrics with tighter SLOs.

D. Batching of requests

Inference requests can be batched together for higher throughput. Several prior works have explored batching [23], [81]. Figure 2 shows the timelines for inference with three common batching mechanisms. The default mechanism only batches at the *request-level* (Figure 2(a)). In this case, ready requests are batched together, but all the forward passes for these requests are completed before any other requests are run. Since requests can have long token generation phases, this can lead to long wait times for requests arriving in between, causing high TTFT and high E2E latencies. An optimization is *continuous batching* [81] (Figure 2(b)). In this case, scheduling decisions are made before each forward pass of the model. However, any given batch comprises either only of requests in their prompt phase or only requests in token phase. Prompt phase is considered more important since it impacts TTFT. Hence, a waiting prompt can preempt a token phase. Although this leads to shorter TTFT, it can substantially increase the tail

请求并在不同的机器上运行它们。这样做可以让我们单独管理每个阶段的硬件资源，从而提高GPU利用率和系统的整体效率。它还允许为每个阶段使用不同的、更适合的硬件。为了实现这样的设置，来自提示计算的缓存上下文需要以低延迟从提示处理机传送到令牌生成机。我们通过当今数据中心可用的后端 Infiniband 互连以优化的方式实现这些传输，从而使我们能够提高效率，而不会造成任何可察觉的性能损失。

通过 Splitwise，我们使用 LLM 推理请求的生产跟踪来设计针对成本、吞吐量和功耗进行优化的集群 [4]。考虑到各代 GPU 的内存和计算扩展率存在差异，我们还评估了不同推理阶段的不同 GPU 和功率上限。这使我们能够为用户提供更好的每美元性能 (Perf/\$)，为 CSP 提供更好的每瓦性能 (Perf/W)。此外，用户可以瞄准较旧的 GPU，因为他们可能更容易使用这些 GPU。

我们表明，基于 Splitwise 的 LLM 推理集群可以以比现有集群低 20% 的成本实现高 1.4× 的吞吐量。或者，它们可以在相同的成本和功耗预算下提供多 2.35× 的吞吐量。

概括。我们做出以下贡献：

- 1) 使用生产跟踪对 NVIDIA A100 和 H100 GPU 上的 LLM 推理中的提示和令牌生成阶段的执行和使用模式差异进行了广泛的表征。
- 2) Splitwise，我们优化利用可用硬件的技术，它将即时计算和令牌生成阶段拆分到不同的机器上。
- 3) 使用 Splitwise 对同构和异构集群部署进行设计探索，以优化总体成本、请求吞吐量和配置功率。
- 4) 使用生产轨迹对使用 Splitwise 设计的系统进行评估。

二. 背景

A. Large Language Models

现代大语言模型是 Transformer 模型使用注意力 [77] 和多层感知器层分别来理解输入并生成输出。基于 Transformer 的 LLM 包括仅编码器 [36]、[54]、仅解码器 [67]、[69]、[71] 和编码器-解码器 [70] 模型。生成式 LLM 是本文的重点，通常是纯解码器模型或编码器-解码器模型。

B. Generative LLM inference phases

图 1 显示了生成式 LLM 推理的示例。一旦收到提示查询，所有输入标记都会在一次迭代内并行计算，以生成第一个标记。我们将此称为即时处理阶段。在提示计算期间从注意层生成的上下文被保存在键值 (KV) 缓存中，因为它

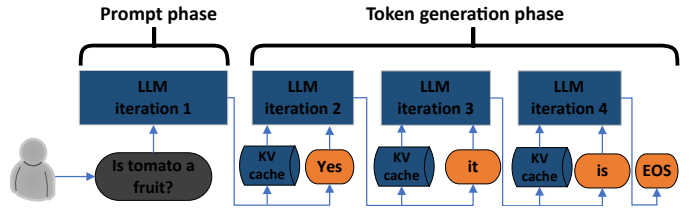


图 1: LLM 推理示例。

Metric	Importance to user
End-to-end (E2E) latency	Total query time that the user sees
Time to first token (TTFT)	How quickly user sees initial response
Time between tokens (TBT)	Average token streaming latency
Throughput	Requests per second

表二：大语言模型性能指标。

所有未来的代币生成迭代都需要。第一个 token 生成后，后续 token 仅使用最后生成的 token 和 KV 缓存作为模型前向传递的输入。这使得后续的令牌生成比计算量大的提示阶段更加密集的内存带宽和容量。

C. Performance metrics for LLMs

之前的工作提出了 LLM 推理的三个主要指标：端到端 (E2E) 延迟、首次令牌时间 (TTFT) 和吞吐量。我们添加了另一个延迟指标：令牌之间的时间 (TBT)，以跟踪令牌连续生成时的在线流吞吐量。表二总结了我们在这项工作中考虑的关键绩效指标。

生成式 LLM 可用于具有不同类型 SLO 的各种任务。对于批处理任务 (e.g., 摘要)，TTFT 或 TBT 延迟指标不如吞吐量重要。另一方面，对于延迟敏感的任务 (e.g., 会话式 API)，TTFT 和 TBT 是具有更严格 SLO 的更重要指标。

D. Batching of requests

推理请求可以批处理在一起以获得更高的吞吐量。之前的几项工作已经探索了批处理 [23]、[81]。图 2 显示了三种常见批处理机制的推理时间表。默认机制仅在 request-level (图 2(a)) 处进行批处理。在这种情况下，就绪请求会一起批处理，但这些请求的所有前向传递都会在任何其他请求运行之前完成。由于请求可能具有较长的令牌生成阶段，因此这可能会导致请求在其间到达的等待时间较长，从而导致较高的 TTFT 和较高的 E2E 延迟。一种优化是 continuous batching [81] (图 2(b))。在这种情况下，调度决策是在模型的每次前向传递之前做出的。然而，任何给定的批次要么仅包含提示阶段的请求，要么仅包含令牌阶段的请求。提示阶段被认为更重要，因为它会影响 TTFT。因此，等待提示可以抢占令牌阶段。虽然这会导致 TTFT 变短，但会大幅增加尾部

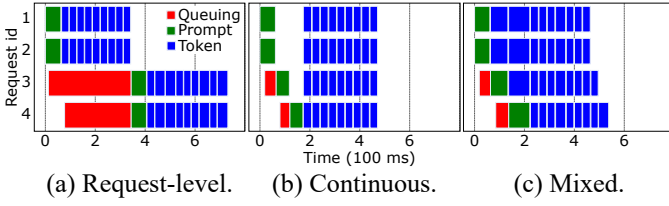


Fig. 2: Batching mechanisms and their latency impact on the prompt and token phases.

for TBT, and therefore the E2E latency. Finally, there is *mixed batching* (Figure 2(c)) [23]. With this batching, the scheduling decisions are made at each forward pass, and the prompt and token phases can run together. This reduces the impact on TBT, but does not eliminate it, since token phases scheduled with prompt phases will experience a longer runtime. In the rest of the paper, we use mixed batching unless stated otherwise.

E. Model parallelism

Model parallelism can be used to divide a model onto multiple GPUs, and even multiple machines, for higher efficiency and memory capacity. LLM inference typically uses pipeline and tensor parallelism. Pipeline parallelism (PP) divides the layers of the model among the GPUs, while keeping all the operators and tensors within a layer on the same GPU. Tensor parallelism (TP) divides the tensor across the GPUs, while replicating all the layers on each GPU. Pipeline parallelism requires lower communication across the participating GPUs, while tensor parallelism requires high bandwidth communication for each layer. In general, tensor parallelism performs better for GPUs within the same machine, connected with high bandwidth interconnects like *e.g.* NVLink [15]. In the rest of the paper, we use tensor parallelism across 8 GPUs for the best latency.

F. GPU clusters and interconnects

With the recent rise of LLM use cases, several cloud service providers have expanded the GPU-based offerings, leading to large GPU cluster deployments [5], [56], [57]. Each machine in these AI clusters is generally comprised of 8 flagship NVIDIA GPUs (A100 or H100). Each GPU is connected to all the other GPUs in the cluster with a high bandwidth Mellanox InfiniBand interconnect [10], [13], forming a high bandwidth data plane network. The InfiniBand bandwidth offered in the cloud today ranges from 25 to 50GBps per GPU pair [7], [10].

III. CHARACTERIZATION

In this section, we explore the performance and utilization characteristics of LLM inference and draw key insights to guide the design of Splitwise.

Production traces. We use production traces taken from two Azure LLM inference services on November 11th 2023. Our traces represent the most common scenarios in LLM inference today: *coding* and *conversation*. We have released a subset of our traces at <https://github.com/Azure/AzurePublicDataset> [4]. The traces we use for characterization are 20 minutes long and include the arrival time, input size (number of prompt tokens),

Model	#Layers	Hidden size	#Heads
Llama2-70B	80	8192	32
BLOOM-176B	70	14336	112

TABLE III: Models we evaluate and their parameters.

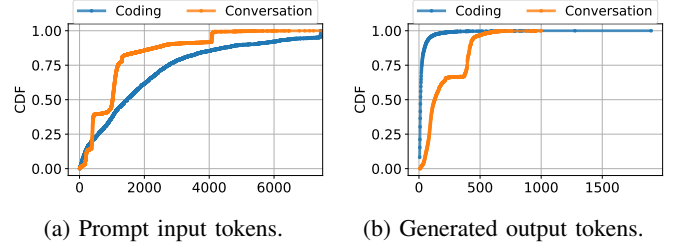


Fig. 3: Distribution for prompt and generated tokens.

and output size (number of output tokens). Due to customer privacy requirements (*e.g.*, GDPR), we do not have visibility into the content of the prompts. We instead use the production traces to guide the input and output sizes, where we send the input prompt with the required number of tokens, and force the model to generate the corresponding number of output tokens for each request. Note that the text of the inputs prompts does not impact the performance metrics that we benchmark, since they depend only on the input and output sizes. For this characterization, we do not reuse the KV-cache between requests to emulate a cloud service with security guarantees.

Models. Table III shows the models that we evaluate. Both BLOOM [69] and Llama2 [71] are state-of-the-art open source LLMs. Both models are decoder-only, transformer-based models. We use the version of each model with the most parameters, since these versions are the most representative for production-class accuracy. Unless stated otherwise, we run BLOOM-176B and Llama-70B on vLLM [51] on a machine with 8 H100 [16] GPUs.

A. Number of prompt and generated tokens

To better understand our traces, we examine the distribution of the number of *prompt input* and *generated output* tokens. Figure 3a shows the distribution of number of prompt tokens. Since the coding LLM inference service is generally used to generate completions as the user is writing code, its input prompt can include large chunks of the code written so far. Thus, it has a large median prompt size of 1500 tokens. On the other hand, the conversation service has a wider range of input prompt tokens since it depends on the user. The median number of prompt tokens for this trace is 1020 tokens.

Figure 3b shows the distribution of the number of generated tokens. Since the coding service typically only generates the next few words in the program as the user types, the median number of output token is 13 tokens. On the other hand, the conversation service has an almost bimodal distribution, with a median of 129 tokens generated.

Insight I: Different inference services may have widely different prompt and token distributions.

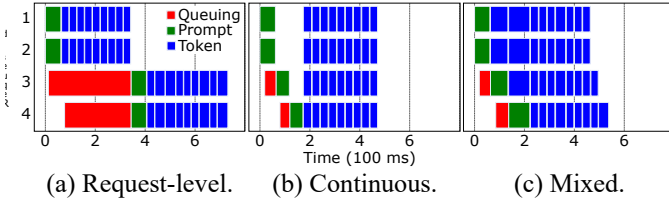


图 2: 批处理机制及其对提示和令牌阶段的延迟影响。

对于 TBT, 因此也有 E2E 延迟。最后是 *mixed batching* (图 2(c)) [23]。通过这种批处理, 调度决策是在每次前向传递时做出的, 并且提示和令牌阶段可以一起运行。这减少了对 TBT 的影响, 但并没有消除它, 因为与提示阶段一起安排的令牌阶段将经历更长的运行时间。在本文的其余部分中, 除非另有说明, 否则我们使用混合配料。

E. Model parallelism

模型并行可用于将模型划分到多个 GPU, 甚至多个机器上, 以获得更高的效率和内存容量。LLM 推理通常使用管道和张量并行性。管道并行性 (PP) 在 GPU 之间划分模型层, 同时将所有运算符和张量保留在同一 GPU 的层内。张量并行性 (TP) 将张量跨 GPU 划分, 同时每个 GPU 上复制所有层。管道并行性要求参与的 GPU 之间的通信量较低, 而张量并行性则需要每层的高带宽通信。一般来说, 张量并行对于同一机器内的 GPU 来说表现更好, 并与 *e.g.* NVLink [15] 等高带宽互连连接。在本文的其余部分中, 我们在 8 个 GPU 上使用张量并行来实现最佳延迟。

F. GPU clusters and interconnects

随着最近 LLM 用例的兴起, 一些云服务提供商扩展了基于 GPU 的产品, 从而导致大型 GPU 集群部署 [5]、[56]、[57]。这些 AI 集群中的每台机器通常由 8 个旗舰 NVIDIA GPU (A100 或 H100) 组成。每个 GPU 通过高带宽 Mellanox InfiniBand 互连 [10]、[13] 连接到集群中的所有其他 GPU, 形成高带宽数据平面网络。目前, 云中提供的 InfiniBand 带宽范围为每个 GPU 对 25 到 50GBps [7]、[10]。

三. 特征描述

在本节中, 我们将探讨 LLM 推理的性能和利用特征, 并得出关键见解来指导 Splitwise 的设计。

生产痕迹。我们使用 2023 年 11 月 11 日th 取自两个 Azure LLM 推理服务的生产跟踪。我们的跟踪代表了当今 LLM 推理中最常见的场景: *coding* 和 *conversation*。我们已在 <https://github.com/Azure/AzurePublicDataset> [4] 上发布了我们的跟踪子集。我们用于表征的轨迹长度为 20 分钟, 包括到达时间、输入大小 (提示标记的数量)、

Model	#Layers	Hidden size	#Heads
Llama2-70B	80	8192	32
BLOOM-176B	70	14336	112

表 III: 我们评估的模型及其参数。

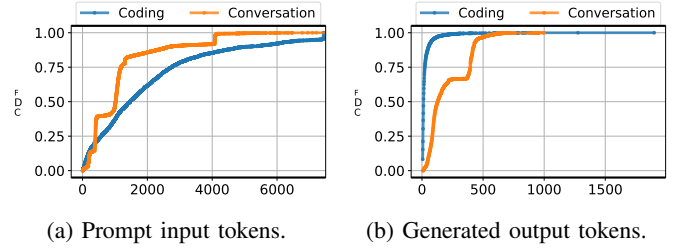


图 3: 提示和生成的令牌分布。

和输出大小 (输出标记的数量)。由于客户隐私要求 (*e.g.*、GDPR), 我们无法查看提示的内容。相反, 我们使用生产跟踪来指导输入和输出大小, 其中我们发送带有所需数量的令牌的输入提示, 并强制模型为每个请求生成相应数量的输出令牌。请注意, 输入提示的文本不会影响我们基准测试的性能指标, 因为它们仅取决于输入和输出大小。对于此特征, 我们不会在请求之间重用 KV 缓存来模拟具有安全保证的云服务。

模型。表 III 显示了我们评估的模型。BLOOM [69] 和 Llama2 [71] 都是最先进的开源大语言模型。这两种模型都是仅解码器、基于变压器的模型。我们使用每个模型参数最多的版本, 因为这些版本最能代表生产级精度。除非另有说明, 我们在具有 8 个 H100 [16] GPU 的机器上在 vLLM [51] 上运行 BLOOM-176B 和 Llama-70B。

A. Number of prompt and generated tokens

为了更好地理解我们的痕迹, 我们检查了 *prompt input* 和 *generated output* 标记数量的分布。图 3a 显示了提示标记数量的分布。由于编码 LLM 推理服务通常用于在用户编写代码时生成补全, 因此其输入提示可能包含迄今为止编写的大量代码。因此, 它的提示大小中值较大, 为 1500 个令牌。另一方面, 会话服务具有更广泛的输入提示标记, 因为它取决于用户。此跟踪的提示标记的中位数是 1020 个标记。

图 3b 显示了生成令牌数量的分布。由于编码服务通常仅在用户键入时生成程序中的接下来的几个单词, 因此输出令牌的中位数为 13 个令牌。另一方面, 对话服务几乎呈双峰分布, 生成的令牌中位数为 129 个。

Insight I: 不同的推理服务可能具有截然不同的提示和令牌分布。

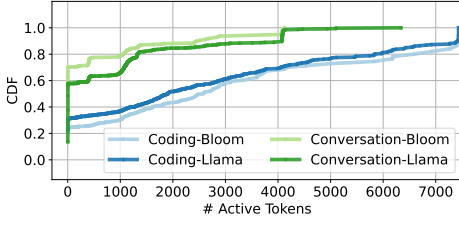


Fig. 4: Cumulative distribution of time spent with various active batched tokens.

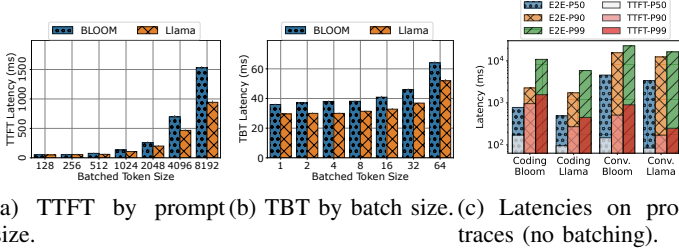


Fig. 5: TTFT, TBT, and E2E for BLOOM-176B and Llama-70B on DGX-H100.

B. Batch utilization

To understand how much can these requests be batched, we measure how often machines run at a given batch size. We use mixed continuous batching as shown in Figure 2. To fit into a single machine, we run a scaled-down version of the coding and conversation traces with 2 requests per second.

Figure 4 shows the distribution of the time spent by the machine running various number of active tokens in a batch. Note that if a prompt of 100 tokens is running in its prompt phase, we count the active tokens as 100. However, once the request is in the token phase, we count it as one active token, since the tokens are generated one at a time (assuming a beam search size of one [51]). We find that most of the time (60–70%) for conversation is spent running only 20 tokens or fewer. Since the coding service has very few output tokens, it experiences even worse batching in the token phase and runs with a single token for more than 20% of the time. Both the LLMs show very similar trends.

Insight II: Mixed continuous batching spends most of the time with very few active tokens batched.

C. Latency

TTFT. Figure 5a shows the impact of the number of prompt tokens on TTFT. The range of sizes was chosen based on the coding and conversation traces. We find that TTFT for both models grows almost linearly as the prompt size increases. This behavior is due to the prompt phase having high GPU utilization and being computationally bound.

TBT. Figure 5b shows the impact of forcefully batching the output tokens of different requests together on the TBT. We observe very little impact on TBT as the batch size grows. With a batch size of 64, there is only 2 $\times$ impact on TBT.

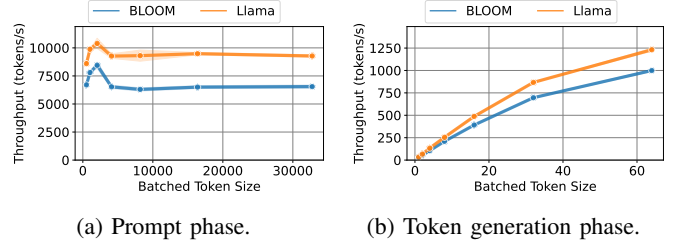


Fig. 6: Impact of batching on the throughput for the 2 LLMs.

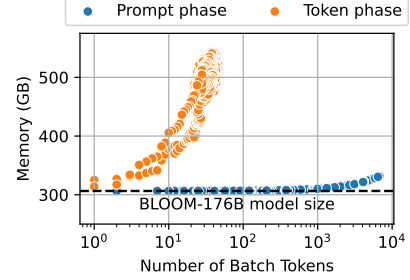


Fig. 7: Required memory with batching in prompt/token phases.

E2E. Figure 5c shows various percentiles of E2E latency for both models, with no batching. The variability between the request input and output sizes is apparent. Furthermore, we see that most of the E2E time is spent running the token phase. This holds true even for the coding trace, where prompt sizes are large and generated tokens few. In fact, we find that for BLOOM-176B, a prompt phase with 1500 input tokens takes the same time as token phase with only 6 output tokens.

Insight III: For most requests, the majority of the E2E time is spent in the token generation phase.

D. Throughput

Figure 6 shows the impact of batching on the throughput (measured as tokens per second). For the prompt phase, we define the throughput as the number of prompt input tokens that are processed per second. We see that the throughput decreases after 2048 prompt tokens, which corresponds to a batch size of less than 2 for the median prompt sizes from the traces. On the other hand, Figure 6b shows that the throughput in the token phase keeps increasing with batching until 64 batch-size, at which point, the machine runs out of memory.

Insight IV: The prompt phase batch size should be limited to ensure good performance. In contrast, batching the token generation phase yields high throughput without any downside.

E. Memory utilization

During an LLM inference, the GPU memory is used to host the model weights and activations, as well as the KV caches (Section II-B). As the number of tokens in a batch increase, the memory capacity required for the KV cache also increases. Figure 7 shows the memory capacity utilization during each phase as the number of tokens in the batch increases. During the prompt phase, the input prompt tokens generate the KV

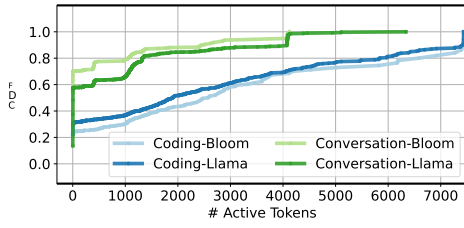
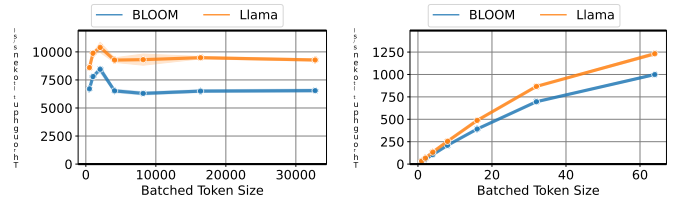


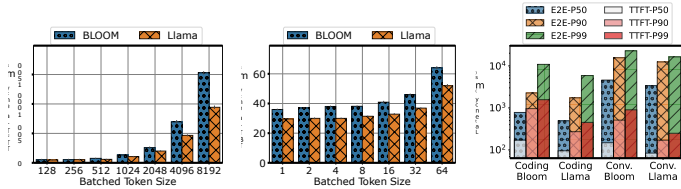
图 4: 各种活跃批量代币花费的时间累积分布。



(a) Prompt phase.

(b) Token generation phase.

图 6: 批处理对 2 个大语言模型的影响。



(a) TTFT by prompt (b) TBT by batch size. (c) Latencies on prod traces (no batching).

图 5: DGX-H100 上 BLOOM-176B 和 Llama-70B 的 TTFT、TBT 和 E2E。

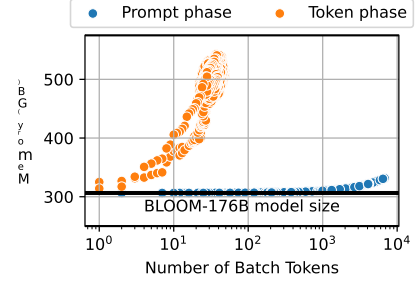


图 7: 提示/令牌阶段批处理所需的内存

B. Batch utilization

为了了解这些请求可以批量处理多少，我们测量机器在给定批量大小下运行的频率。我们使用混合连续批处理，如图 2 所示。为了适应单台机器，我们以每秒 2 个请求运行缩小版本的编码和对话跟踪。

图 4 显示了机器在一批中运行不同数量的活动令牌所花费的时间分布。请注意，如果在提示阶段运行包含 100 个令牌的提示，则我们将活动令牌计为 100。但是，一旦请求处于令牌阶段，我们将其计为一个活动令牌，因为令牌一次生成一个（假设波束搜索大小为 1 [51]）。我们发现对话的大部分时间 (60-70%) 仅花费在 20 个或更少的令牌上。由于编码服务的输出令牌非常少，因此它在令牌阶段的批处理效果更差，并且超过 20% 的时间使用单个令牌运行。两个大语言模型表现出非常相似的趋势。

Insight II: 混合连续批处理大部分时间都只处理很少的活跃令牌。

C. Latency

薄膜晶体管。图 5a 显示了提示令牌数量对 TTFT 的影响。大小范围是根据编码和对话痕迹选择的。我们发现，随着提示大小的增加，两种模型的 TTFT 几乎呈线性增长。此行为是由于提示阶段具有较高的 GPU 利用率并且受到计算限制。

TBT。图 5b 显示了将不同请求的输出令牌强制批处理在一起对 TBT 的影响。我们发现，随着批量大小的增加，对 TBT 的影响非常小。批量大小为 64 时，对 TBT 的影响仅为 2x。

端到端。图 5c 显示了两种模型的 E2E 延迟的各个百分位数（没有批处理）。请求输入和输出大小之间的变化是显而易见的。此外，我们看到大部分 E2E 时间都花在运行令牌阶段。即使对于提示大小很大且生成的标记很少的编码跟踪也是如此。事实上，我们发现对于 BLOOM-176B，具有 1500 个输入令牌的提示阶段与仅具有 6 个输出令牌的令牌阶段花费的时间相同。

Insight III: 对于大多数请求，大部分 E2E 时间都花费在令牌生成阶段。

D. Throughput

图 6 显示了批处理对吞吐量的影响（以每秒令牌数衡量）。对于提示阶段，我们将吞吐量定义为每秒处理的提示输入令牌的数量。我们看到吞吐量在 2048 个提示标记后下降，这对应于跟踪中提示大小中位数小于 2 的批量大小。另一方面，图 6b 显示令牌阶段的吞吐量随着批处理不断增加，直到 64 个批处理大小，此时机器会耗尽内存。

Insight IV: 应限制提示阶段批量大小以确保良好的性能。相比之下，对令牌生成阶段进行批处理可以产生高吞吐量，而且没有任何缺点。

E. Memory utilization

在 LLM 推理期间，GPU 内存用于托管模型权重和激活以及 KV 缓存（第 II-B 节）。随着批次中 token 数量的增加，KV 缓存所需的内存容量也会增加。图 7 显示了随着批次中令牌数量的增加，每个阶段的内存容量利用率。在提示阶段，输入提示 token 生成 KV

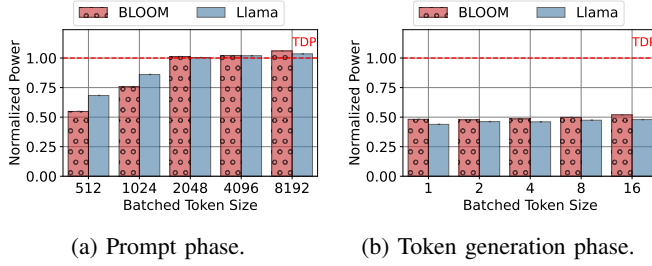


Fig. 8: Maximum and mean power utilization varying the batching size.

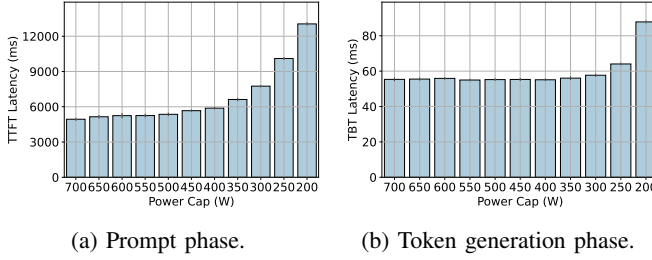


Fig. 9: Impact of power cap on the prompt and token generation latency with the maximum batch size possible.

cache. During the output token phase, *each* active generated token that is being processed accesses the KV cache of its *entire context* so far.

Insight V: Batching during the prompt phase is compute-bound, whereas the token phase is limited by memory capacity.

F. Power utilization

When hosting machines, cloud providers need to consider the peak power draw, which has direct impact in the datacenter cost [26]. This is especially important when building GPU clusters, since GPUs consume much higher power than regular compute machines [63], [64]. Figure 8 shows the GPU power draw normalized to the thermal design power (TDP) when running prompt and token generation phases. Since the prompt phase is compute intensive, its power draw increases with batch size. On the other hand, the token phase is memory bound and its power draw does not vary when increasing the number of tokens to process.

Providers can cap the power usage of the machines to reduce the peak power. Figure 9 shows the impact to latency when increasing the power caps for both prompt and token phases. The prompt phase is highly sensitive to the power cap and the latency increases substantially. On the other hand, the token generation phase incurs almost no latency impact when power capping by over 50% (*i.e.*, 700 to 350W).

Insight VI: While the prompt phase utilizes the power budget of the GPU efficiently, the token phase does not.

G. GPU hardware variations

Given the different characteristics of prompt and token generation phases, we measure the performance impact on

	Coding			Conversation		
	A100	H100	Ratio	A100	H100	Ratio
TTFT	185 ms	95 ms	0.51×	155 ms	84 ms	0.54×
TBT	52 ms	31 ms	0.70×	40 ms	28 ms	0.70×
E2E	856 ms	493 ms	0.58×	4957 ms	3387 ms	0.68×
Cost [5]	\$0.42	\$0.52	1.24×	\$2.4	\$3.6	1.5×
Energy	1.37 Whr	1.37 Whr	1×	7.9 Whr	9.4 Whr	1.2×

TABLE IV: P50 request metrics on A100 vs. H100 without batching on Llama-70B.

the two from running on different hardware. Table I shows the specifications for DGX-A100 [15] and DGX-H100 [16]. The memory-to-compute ratio favors A100 over H100. Table IV shows our findings. We see a lower performance impact on the token generation phase (TBT) as compared to the Prompt phase (TTFT). Since coding requests are dominated by prompt phase, by having very few generated tokens, the E2E latency impact from A100 is worse on coding than conversation. Furthermore, we see that A100 has better or equal inference cost and energy overall compared to H100.

Insight VII: Token generation can be run on less compute-capable hardware for better Perf/W and Perf/\$ efficiencies.

IV. SPLITWISE

Based on our characterization insights, we propose Splitwise, a technique to split the prompt and generation phases in the LLM inference on to separate machines.

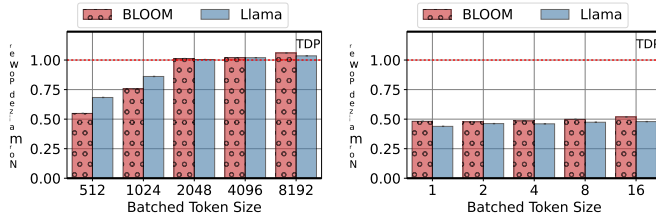
Figure 10 shows the high-level overview of Splitwise. We maintain two separate pools of machines for prompt and token processing. A third machine pool, the mixed pool, expands and contracts as needed by the workload. All machines are pre-loaded with the model of choice. When a new inference request arrives, the scheduler allocates it to a pair of machines (*i.e.*, prompt and token). The prompt machines are responsible for generating the first token for an input query, by processing all the input prompt tokens in the prompt phase and generating the KV-cache. The prompt machine also sends over the KV-cache to the token machine, which continues the token generation until the response is complete. We use continuous batching at the token machines to maximize their utilization. Machines in mixed pool use mixed continuous batching.

At a lower request rate, we target better latency in Splitwise, while, at a higher request rate, we target avoiding any performance or throughput reduction due to the fragmentation between prompt and token machine pools.

Splitwise uses a hierarchical two-level scheduling as shown in Figure 10. The cluster-level scheduler (CLS) ① is responsible for machine pool management and for routing incoming inference requests. The machine-level scheduler (MLS) ② maintains the pending queue and manages batching of requests at each machine.

A. Cluster-level scheduling

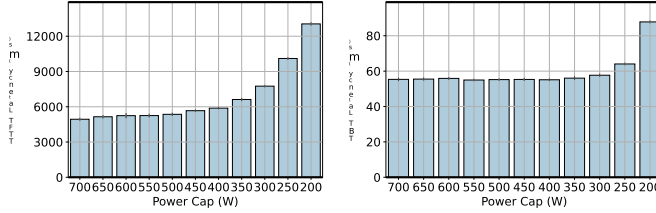
Machine pool management. The CLS maintains the prompt, token, and mixed machine pools ③. Splitwise initially assigns



(a) Prompt phase.

(b) Token generation phase.

图 8: 随批量大小变化的最大和平均功率利用率。



(a) Prompt phase.

(b) Token generation phase.

图 9: 在可能的最大批量大小下, 功率上限对提示和令牌生成延迟的影响。

缓存。在输出令牌阶段, 正在处理的 $each$ 主动生成的令牌到目前为止访问其 $entire$ context的KV缓存。

Insight V: 提示阶段的批处理受计算限制, 而令牌阶段则受内存容量的限制。

F. Power utilization

在托管机器时, 云提供商需要考虑峰值功耗, 这对数据中心成本有直接影响[26]。这在构建 GPU 集群时尤其重要, 因为 GPU 比常规计算机消耗更高的功率 [63]、[64]。图 8 显示了运行提示和令牌生成阶段时标准化为热设计功耗 (TDP) 的 GPU 功耗。由于提示阶段是计算密集型的, 因此其功耗随着批量大小而增加。另一方面, 令牌阶段受内存限制, 并且当增加要处理的令牌数量时, 其功耗不会变化。

提供商可以限制机器的用电量以降低峰值功率。图 9 显示了增加提示阶段和令牌阶段的功率上限对延迟的影响。提示阶段对功率上限高度敏感, 并且延迟大幅增加。另一方面, 当功率上限超过 50% (*i.e.*, 700 至 350W) 时, 令牌生成阶段几乎不会产生延迟影响。

Insight VI: 虽然提示阶段有效地利用了 GPU 的功率预算, 但令牌阶段却没有。

G. GPU hardware variations

考虑到提示和令牌生成阶段的不同特征, 我们衡量了对性能的影响

	Coding			Conversation		
	A100	H100	Ratio	A100	H100	Ratio
TTFT	185 ms	95 ms	0.51×	155 ms	84 ms	0.54×
TBT	52 ms	31 ms	0.70×	40 ms	28 ms	0.70×
E2E	856 ms	493 ms	0.58×	4957 ms	3387 ms	0.68×
Cost [5]	\$0.42	\$0.52	1.24×	\$2.4	\$3.6	1.5×
Energy	1.37 Whr	1.37 Whr	1×	7.9 Whr	9.4 Whr	1.2×

表 IV: A100 与 H100 上的 P50 请求指标, 无需在 Llama-70B 上进行批处理。

两者运行在不同的硬件上。表 I 显示了 DGX-A100 [15] 和 DGX-H100 [16] 的规格。内存与计算的比率有利于 A100, 而不是 H100。表四显示了我们的发现。与提示阶段 (TTFT) 相比, 我们发现令牌生成阶段 (TBT) 的性能影响较低。由于编码请求以提示阶段为主, 生成的令牌非常少, 因此 A100 对编码的端到端延迟影响比对话更严重。此外, 我们发现与 H100 相比, A100 总体上具有更好或相同的推理成本和能量。

Insight VII: 令牌生成可以在计算能力较低的硬件上运行, 以获得更好的 Perf/W 和 Perf/\$ 效率。

四. 分裂

基于我们的表征见解, 我们提出了 Splitwise, 一种将 LM 推理中的提示阶段和生成阶段拆分到不同机器上的技术。

图 10 显示了 Splitwise 的高级概述。我们维护两个独立的机器池用于提示和令牌处理。第三个机器池, 即混合池, 可根据工作负载的需要进行扩展和收缩。所有机器均预装了所选型号。当新的推理请求到达时, 调度程序将其分配给一对机器 (*i.e.*、提示符和令牌)。提示机负责通过处理提示阶段的所有输入提示令牌并生成 KV 缓存来生成输入查询的第一个令牌。提示机还将 KV 缓存发送到令牌机, 令牌机继续生成令牌直到响应完成。我们在代币机上使用连续批处理来最大限度地提高其利用率。混合池机采用混合连续配料。

在较低的请求率下, 我们的目标是在 Splitwise 中实现更好的延迟, 而在较高的请求率下, 我们的目标是避免由于提示和令牌机器池之间的碎片而导致任何性能或吞吐量下降。

Splitwise 使用分层两级调度, 如图 10 所示。集群级调度程序 (CLS) 1 负责机器池管理和路由传入的推理请求。机器级调度程序 (MLS) 2 维护待处理队列并管理每台机器上的请求批处理。

A. Cluster-level scheduling

机器池管理。CLS 维护提示、令牌和混合机器池 3。Splitwise 初始分配

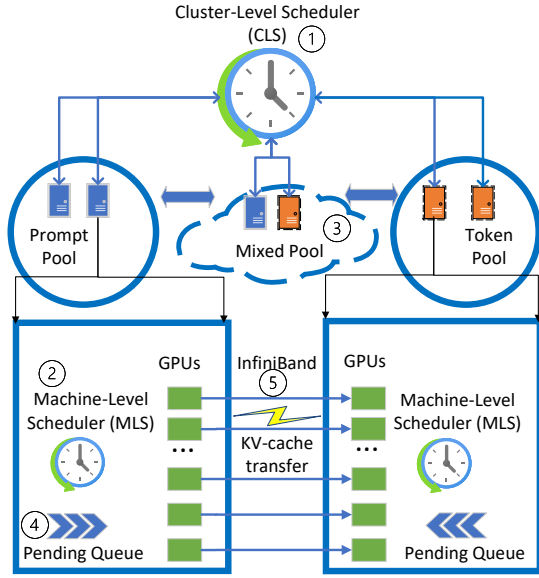


Fig. 10: High-level system diagram of Splitwise.

machines to the prompt or token pool depending on the expected request load and input/output token distributions. Machines from the prompt or token pools may be dynamically moved into and out of the mixed pool to reduce fragmentation and meet SLOs at higher loads. A machine in the mixed pool retains its identity as a prompt or token machine and goes back to its original pool once there are no tasks of the opposite kind in its pending queue. Switching pools does not incur any noticeable latency. If the load distribution deviates considerably from initial assumptions, Splitwise employs a coarse grained *re-purposing of machines* and moves machines between the prompt and token pools. Re-purposing of machines is done infrequently, typically only if they stay in the mixed pool for a considerable amount of time.

Request routing. CLS uses Join the Shortest Queue (JSQ) scheduling [39], [85] to assign a prompt and a token machine to each request. Queue lengths are defined by the number of pending tokens. Each machine regularly communicates to the CLS changes in its memory capacity or pending queue. Note that this does not happen at every iteration boundary. We simultaneously assign both the prompt and token machine when scheduling requests, since we can then overlap KV-cache transfers with prompt computation to reduce transfer overheads (Section IV-C).

When routing requests, if the pending queue is bigger than a certain threshold, the CLS looks for target machines in the mixed pool. If the mixed pool is also full, it proceeds to look in the opposite pool (*i.e.*, a token machine to run prompts and vice versa) and moves the machine into the mixed pool. Machines in the mixed pool operate exactly as a non-Splitwise machine would, with mixed batching. Once the queue of mixed requests is drained, the CLS transitions the machine back to its original pool. For example, when the queue is too long, we can move a prompt machine to the mixed pool to run tokens;

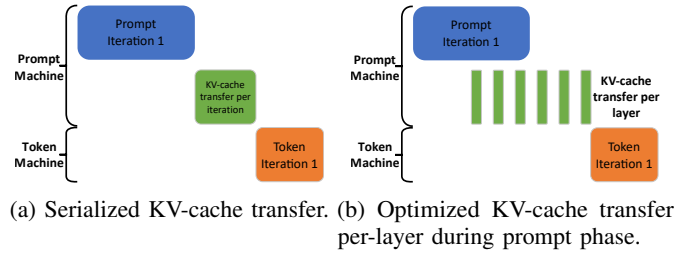


Fig. 11: Optimizing KV-cache transfer in Splitwise.

once the machine is done running tokens, we transition the machine back into the prompt pool.

B. Machine-level scheduling

The MLS runs on each machine and is responsible for tracking the GPU memory utilization, maintaining the pending queue (4), deciding the batch for each iteration, and reporting the relevant status to the CLS.

Prompt machines. The MLS simply uses first-come-first-serve (FCFS) to schedule prompts. The results in Figure 6a show that after 2048 prompt tokens, the throughput degrades. For this reason, the MLS restricts the batching of multiple prompts together to 2048 tokens in total. This is a configurable value, and can change for a different model or hardware.

Token machines. The MLS uses FCFS to schedule tokens and batches as much as possible. Figure 6b shows that the token generation throughput keeps scaling up with the batch size until the machine runs out of memory. For this reason, the MLS tracks the memory and starts queueing tokens once the machine is close to running out of memory.

Mixed machines. To meet the TTFT SLO, the MLS must prioritize running prompts and schedule any new prompts in the pending queue immediately. If the machine is running token phases and has no additional capacity to run the prompt phase, the MLS will *preempt* tokens. To avoid *starvation* of the token phase due to preemption, we increase the priority of the token with age and limit the number of preemptions that each request can have.

C. KV-cache transfer

As discussed in Section II, the KV-cache is generated during the prompt phase of the request, and it continuously grows during the token generation phase. In Splitwise, we need to transfer the KV-cache from the prompt machine to the token machine (5) (shown in Figure 10) to complete the inference. This transfer delay is the main overhead associated with Splitwise. In this section, we discuss the impact of KV-cache transfer and how we optimize it.

Figure 11a shows the Gantt chart for the prompt phase, the KV-cache transfer, and the token generation phase for a single batch of requests when naively transferring the KV cache in a serialized way. The KV-cache transfer starts only after the prompt phase has finished and the first token is generated. Further, it needs to complete before the next output token

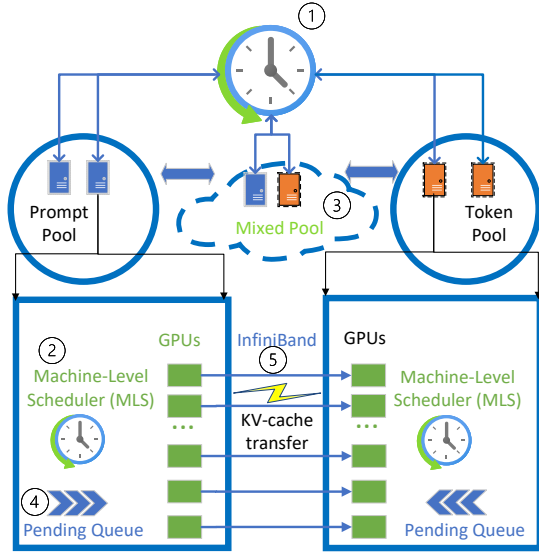


图 10: Splitwise 的高级系统图。

根据预期的请求负载和输入/输出令牌分配，将计算机添加到提示或令牌池。提示池或令牌池中的机器可以动态移入和移出混合池，以减少碎片并满足较高负载下的 SLO。混合池中的机器保留其作为提示机或令牌机的身份，并在其待处理队列中没有相反类型的任务时返回其原始池。切换池不会产生任何明显的延迟。如果负载分布与初始假设有很大偏差，Splitwise 会采用粗粒度 *re-purposing of machines* 并在提示池和令牌池之间移动机器。机器的重新利用很少进行，通常只有当它们在混合池中停留相当长的时间时才会进行。

请求路由。CLS 使用加入最短队列 (JSQ) 调度 [39]、[85] 为每个请求分配提示和令牌机。队列长度由待处理令牌的数量定义。每台机器定期向 CLS 传达其内存容量或挂起队列的变化。请注意，这不会在每次迭代边界发生。我们在调度请求时同时分配提示和令牌机，因为我们可以将 KV 缓存传输与提示计算重叠以减少传输开销（第 IV-C 节）。

在路由请求时，如果待处理队列大于某个阈值，CLS 会在混合池中查找目标计算机。如果混合池也满了，它会继续查找相反的池（*i.e.*，一个运行提示的令牌机器，反之亦然）并将该机器移动到混合池中。混合池中的机器的运行方式与非 Splitwise 机器完全相同，采用混合配料。一旦混合请求队列耗尽，CLS 就会将计算机转换回其原始池。例如，当队列太长时，我们可以将提示机移至混合池中运行代币；

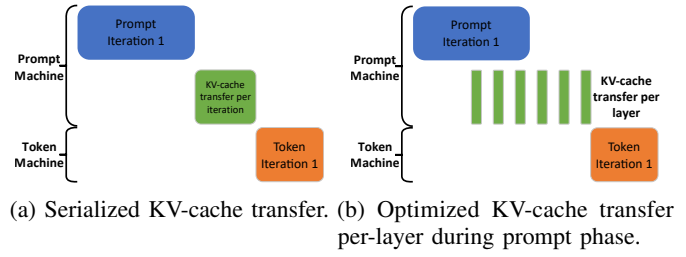


图 11: 优化 Splitwise 中的 KV 缓存传输。

一旦机器完成运行令牌，我们就将机器转换回提示池。

B. Machine-level scheduling

MLS 在每台机器上运行，负责跟踪 GPU 内存利用率、维护待处理队列 4、决定每次迭代的批次，并向 CLS 报告相关状态。

提示机。MLS 仅使用先到先服务 (FCFS) 来安排提示。图 6a 中的结果显示，在 2048 个提示令牌之后，吞吐量会下降。因此，MLS 将多个提示的批处理限制为总共 2048 个令牌。这是一个可配置的值，可以针对不同的型号或硬件进行更改。

代币机。MLS 使用 FCFS 尽可能地调度令牌和批次。图 6 b 显示令牌生成吞吐量随着批量大小不断扩大，直到机器内存耗尽。因此，MLS 会跟踪内存，并在机器即将耗尽内存时开始对令牌进行排队。

混合机。为了满足 TTFT SLO，MLS 必须优先考虑运行提示并立即在待处理队列中安排任何新提示。如果机器正在运行令牌阶段并且没有额外的能力来运行提示阶段，则 MLS 将 *preempt* 令牌。为了避免由于抢占而导致令牌阶段的 *starvation*，我们随着年龄的增长而增加令牌的优先级，并限制每个请求可以拥有的抢占次数。

C. KV-cache transfer

正如第二节中所讨论的，KV 缓存是在请求的提示阶段生成的，并且在令牌生成阶段不断增长。在 Splitwise 中，我们需要将 KV-cache 从提示机传输到令牌机（如图 10 所示）来完成推理。这种传输延迟是与 Splitwise 相关的主要开销。在本节中，我们讨论 KV 缓存传输的影响以及如何优化它。

图 11a 显示了以序列化方式简单传输 KV 缓存时单批请求的提示阶段、KV 缓存传输和令牌生成阶段的甘特图。KV 缓存传输仅在提示阶段完成并生成第一个令牌后开始。此外，它需要在下一个输出令牌之前完成

can be generated in the token generation phase. This directly impacts the maximum TBT and end-to-end latency of inference.

The time required for the transfer depends on the size of the KV cache (which is directly proportional to the number of prompt tokens) and on the bandwidth of the interconnect between the prompt and the token machines. Even when using fast InfiniBand links, the transfer overhead for large prompt sizes could become a significant fraction of the TBT.

In Splitwise, we optimize the KV-cache transfer by overlapping it with the computation in the prompt phase. As each layer in the LLM gets calculated in the prompt machine, the KV cache corresponding to that layer is also generated. At the end of each layer, we trigger an asynchronous transfer of the KV-cache for that layer while the prompt computation continues to the next layer. Figure 11b shows this asynchronous transfer which reduces the transfer overheads. Layer-wise transfer also enables other optimizations, such as earlier start of the token phase in the token machines, as well as earlier release of KV-cache memory on the prompt machines.

Layer-wise KV-cache transfer happens in parallel with the prompt computation for the next layer. This requires fine-grained synchronization per layer for correctness. Thus, it is possible to incur performance interference and increase the TTFT, especially for smaller prompts. However, for small prompts the total KV-cache size is small and does not need the layer-wise transfer to hide the latency. Since the number of tokens in a batch is already known at the start of computation, Splitwise picks the best technique for KV-cache transfer. It uses serialized KV-cache transfer for smaller prompts and layer-wise transfer and for larger prompts. We show that the overall transfer and interference overheads are relatively small in Section VI-A.

D. Provisioning with Splitwise

We leverage Splitwise to optimize LLM inference cluster deployments for power, cost, and throughput.

Type of machines. We propose four main variants of Splitwise-based systems: *Splitwise-AA*, *Splitwise-HH*, *Splitwise-HA*, and *Splitwise-HHcap*. The nomenclature is simply drawn from the first letter representing the Prompt machine type, and the second letter representing the Token machine type. “A” represents a DGX-A100 machine, “H” represents a DGX-H100 machine, and “Hcap” represents a power-capped DGX-H100 machine. Table V shows a summary of the cost, power, and hardware in each of our evaluated systems.

Splitwise-AA uses DGX-A100 for both prompt and token pools, while Splitwise-HH uses DGX-H100 for both. These two variants represent the commonly available setups in providers where machines are homogeneous and interchangeable.

Splitwise-HA uses DGX-H100 for the prompt pool and DGX-A100 for the token pool. We choose this configuration based on Table IV, and the Insight VII (i.e., A100s can be more cost- and power-efficient for the token phase).

Splitwise-HHcap uses DGX-H100 machines for both prompt and token pools. However, we power cap the token machines down to 70% of their rated power, with each GPU capped by

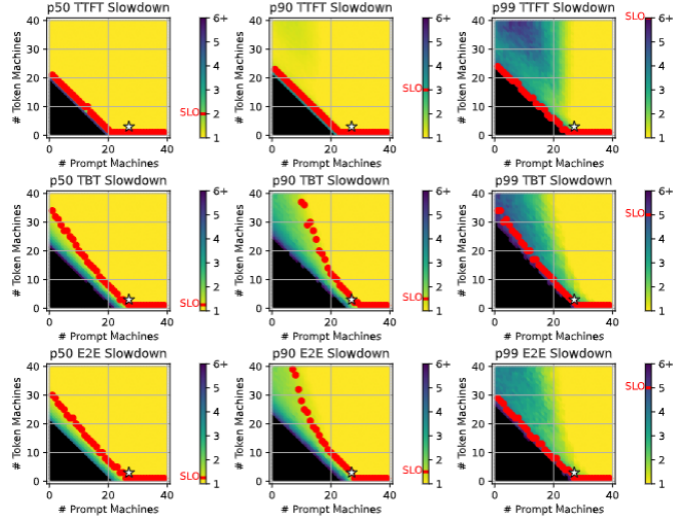


Fig. 12: Design space for provisioning a Splitwise-HH cluster. Cluster configurations targets a peak throughput of 70 RPS. The cost-optimal Splitwise-HH configuration is marked with * (27 prompt and 3 token machines).

50% of the power. We propose this design based on Figure 9 and Insight VII (i.e., the prompts phase is impacted by power caps while token has no performance impact with 50% lower power cap per GPU).

Number of machines. The LLM inference cluster deployment must be sized with the appropriate number of prompt and token machines. Our methodology involves searching the design space using our event-driven cluster simulator, which is described in detail in Section V. We need to provide as input: (1) the target cluster design (e.g., Splitwise-HA or Splitwise-HHcap), (2) an LLM-specific performance model that can estimate the TTFT and TBT at various input, output, and batch sizes, (3) a short trace derived from the target prompt and token size distributions for the service (e.g., Figure 3), (4) the SLOs (e.g., Table VI), (5) the constraints (e.g., throughput), and (6) the optimization goal (e.g., minimize cost). Using this information, our provisioning framework searches the space for the desired optimal point. For example, searching with a throughput constraint and a cost minimization goal gives us iso-throughput cost-optimized clusters across different designs.

Search space. Figure 12 shows an example of the two-dimensional search space for the number of prompt and token machines under Splitwise-HH for the coding workload (using a 2-minute trace). The simulator outputs the various percentiles for TTFT, TBT, and E2E latencies. Then, we select the clusters that meet the SLOs for each of these metrics and optimize our target function. For example, Figure 12 shows a * for the setup with 27 prompt and 3 token machines with the lowest cost that achieves 70 RPS. We call this setup *iso-throughput cost-optimized*.

Optimization. We can use three optimization goals: *throughput*, *cost*, and *power*. Throughput optimization is important for both,

可以在令牌生成阶段生成。这直接影响推理的最大 TBT 和端到端延迟。

传输所需的时间取决于 KV 缓存的大小（与提示令牌的数量成正比）以及提示和令牌机之间互连的带宽。即使使用快速 InfiniBand 链路，大提示大小的传输开销也可能成为 TBT 的很大一部分。

在 Splitwise 中，我们通过将 KV 缓存传输与提示阶段的计算重叠来优化 KV 缓存传输。当 LLM 中的每一层在提示机中计算时，也会生成该层对应的 KV 缓存。在每一层的末尾，我们触发该层的 KV 缓存的异步传输，同时提示计算继续到下一层。图 11b 显示了这种异步传输，它减少了传输开销。逐层传输还可以实现其他优化，例如令牌机中令牌阶段的早期开始，以及提示机上 KV 缓存内存的早期释放。

逐层 KV 缓存传输与下一层的即时计算并行发生。这需要每层进行细粒度同步以确保正确性。因此，可能会产生性能干扰并增加 TTFT，特别是对于较小的提示。然而，对于小提示，总的 KV 缓存大小很小，不需要分层传输来隐藏延迟。由于在计算开始时就已经知道批次中的令牌数量，因此 Splitwise 选择最佳的 KV 缓存传输技术。它使用序列化 KV 缓存传输来处理较小的提示和分层传输以及较大的提示。我们在第 VI-A 节中表明，总体传输和干扰开销相对较小。

D. Provisioning with Splitwise

我们利用 Splitwise 来优化 LLM 推理集群部署的功耗、成本和吞吐量。

机器类型。我们提出了基于 Splitwise 的系统的四种主要变体：Splitwise-AA、Splitwise-HH、Splitwise-HA 和 Splitwise-HHcap。命名法简单地取自代表 Prompt 机器类型的第一个字母和代表 Token 机器类型的第二个字母。“A”代表 DGX-A100 机器，“H”代表 DGX-H100 机器，“Hcap”代表功率上限的 DGX-H100 机器。表 V 显示了我们评估的每个系统的成本、功耗和硬件的摘要。

Splitwise-AA 将 DGX-A100 用于提示池和代币池，而 Splitwise-HH 将 DGX-H100 用于两者。这两种变体代表了供应商中常用的设置，其中机器是同质且可互换的。

Splitwise-HA 使用 DGX-H100 作为提示池，使用 DGX-A100 作为令牌池。我们根据表 IV 和 Insight VII (*i.e.*, A100 在令牌阶段更具成本效益和能效) 选择此配置。

Splitwise-HHcap 使用 DGX-H100 机器作为提示池和代币池。然而，我们将代币机的功率上限降至其额定功率的 70%，每个 GPU 的上限为

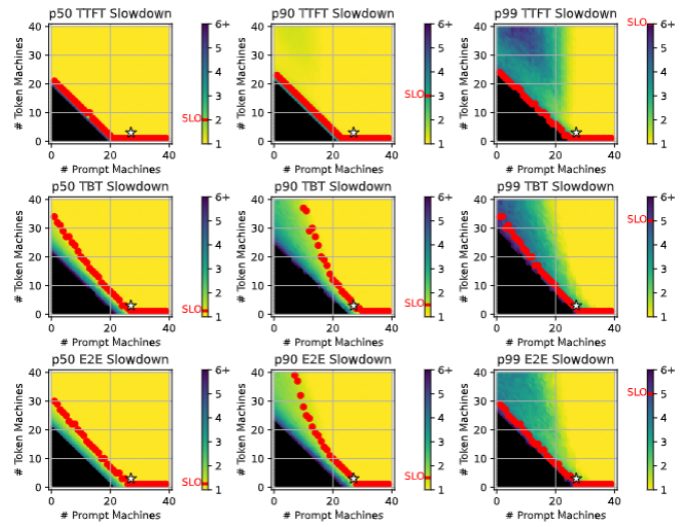


图 12: 配置 Splitwise-HH 集群的设计空间。集群配置的目标是 70 RPS 的峰值吞吐量。成本最优的 Splitwise-HH 配置标有 * (27 提示符和 3 个令牌机)。

50% 的力量。我们基于图 9 和 Insight VII 提出此设计 (*i.e.*, 提示阶段受功率上限影响，而令牌对性能没有影响，每个 GPU 的功率上限降低 50%)。

机器数量。LLM 推理集群部署的规模必须具有适当数量的提示机和令牌机。我们的方法涉及使用事件驱动集群模拟器搜索设计空间，这在第五节中有详细描述。我们需要提供以下输入：(1) 目标集群设计 (*e.g.*, Splitwise-HA 或 Splitwise-HHcap)，(2) 特定于 LLM 的性能模型，可以估计各种输入、输出和批量大小下的 TTFT 和 TBT，(3) 从服务的目标提示和令牌大小分布导出的短跟踪 (*e.g.*, 图 3)、(4) SLO (*e.g.*, 表 VI)、(5) 约束 (*e.g.*, 吞吐量)，以及 (6) 优化目标 (*e.g.*, 最小化成本)。使用此信息，我们的配置框架在空间中搜索所需的最佳点。例如，使用吞吐量约束和成本最小化目标进行搜索为我们提供了跨不同设计的等吞吐量成本优化集群。

搜索空间。图 12 显示了针对编码工作负载（使用 2 分钟跟踪）的 Splitwise-HH 下提示和令牌机数量的二维搜索空间示例。模拟器输出 TTFT、TBT 和 E2E 延迟的各个百分位数。然后，我们选择满足每个指标的 SLO 的集群并优化我们的目标函数。例如，图 12 显示了具有 27 个提示和 3 个令牌机的设置的 *，以最低成本实现 70 RPS。我们将此设置称为 *iso-throughput cost-optimized*。

优化。我们可以使用三个优化目标：throughput、cost 和 power。吞吐量优化对两者都很重要，

	Prompt Machine			Token Machine			Prompt-Token	
	Type	Cost	Power	Type	Cost	Power	Interconnect	Bandwidth
Splitwise-AA	DGX-A100	1×	1×	DGX-A100	1×	1×	1×	
Splitwise-HH	DGX-H100	2.35×	1.75×	DGX-H100	2.5×	1.75×	2×	
Splitwise-HHcap	DGX-H100	2.35×	1.75×	DGX-H100	2.5×	1.23×	2×	
Splitwise-HA	DGX-H100	2.35×	1.75×	DGX-A100	1×	1×	1×	

TABLE V: Evaluated Splitwise designs all normalized to DGX-A100

the cloud service provider (CSP) and the user. Cost optimization has different importance levels to the CSP and the user. For the CSP, a higher cost for the same throughput might be acceptable if there are gains in power and space requirements for the cluster. However, for the end-user, a higher cost at the same throughput is generally unacceptable. Finally, power optimization is attractive for a CSP, since it enables more GPUs to be deployed in the same datacenter [62], [63], but it may not be as important to the user. We only consider the provisioned power, and not the dynamic power utilization, in our study.

E. Practical Considerations

Accuracy impact. Splitwise does not impact accuracy since it uses lossless KV-cache transfer and does not add any randomization. It executes inference with the same parameters and state as on a single machine.

Scalability. Since LLM requests are much longer than typical ML requests [37], [38], they incur lower scheduling overhead for similar cluster sizes. However, the CLS may become a scalability bottleneck for large clusters. Insights from prior work on partitioned or replicated scheduling could help improve scalability [27], [61], [72] and are orthogonal to Splitwise.

Reliability and fault tolerance. If the prompt or the token machine fail, Splitwise simply restarts requests from scratch, similar to today’s LLM serving systems [44], [51]. Alternatively, Splitwise could checkpoint the KV-cache generated after prompt computation into an in-memory database. To recover, Splitwise can use this cache to skip prompt recomputation, and start right away with the token phase. The KV-cache could also be checkpointed periodically during the token phase. Designing safe and efficient failure recovery is out of scope for our paper.

V. METHODOLOGY

A. Experimental setup

To evaluate our proposal on real hardware, we implement Splitwise’s KV-cache transfer mechanism on top of vLLM [51]. Our implementation is open source [1]. We run this modified vLLM on two DGX-A100 and two DGX-H10 virtual machines (VMs) on Microsoft Azure with specifications from Table I. These are the VMs used to collect the characterization data in Section III. These machines are connected with InfiniBand and the DGX-H100s have double the bandwidth (*i.e.*, 400 Gbps).

Since vanilla vLLM only supports continuous batching with token preemption which can lead to much higher TBT, we implement state-of-the-art mixed continuous batching [81] as discussed earlier in Figure 2(c).

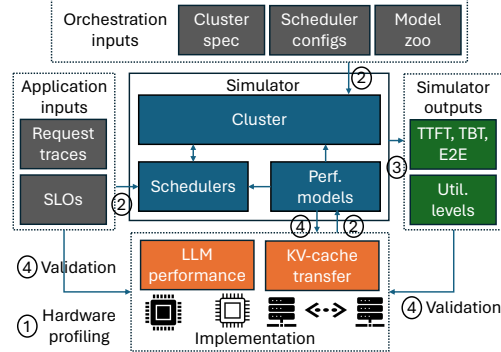


Fig. 13: Overview of the design of the Splitwise simulator.

Our implementation of the Splitwise technique assigns machines either a prompt role, or a token role. As the prompt machine generates the first token, it transfers the KV-cache to the token machine using the technique described in Section IV-C. We use MSCCL++ [11], an optimized GPU-driven communication library, to implement the naive and layer-wise KV cache transfers.

In our implementation, the prompt machine uses the zero-copy one-sided `put` primitive of MSCCL++ to send KV-cache data over InfiniBand as soon as it is ready, without requiring the token machine to issue any receive instructions. Once we have issued a `put` for all layers, the prompt machine signals a semaphore that the token machine waits on. The synchronization done with the help of semaphores uses the same InfiniBand connection used to send KV-cache data. When processing a batch of prompts, each request is assigned a different semaphore since it may be routed to different token machines. We ship the KV-caches block-by-block in vLLM. To minimize the number of transfers, we also consider the contiguity of KV blocks as long as they use the same semaphore.

B. Simulator setup

We build a simulator to explore cluster designs and evaluate Splitwise at scale. The simulator code is open source [20].

Figure 13 shows the design of our simulator. The simulator is event-driven and faithfully models the Splitwise machine pools, schedulers, machine-level memory and queues, and KV-cache transfer. We first profile the LLM on the target hardware with various input/output sizes (1). Based on the characterization profiles, we build a performance model. The simulator takes as input the request traces, SLOs, the performance model, and the configurations for cluster and scheduler (2). For our

	Prompt Machine			Token Machine			Prompt-Token	
	Type	Cost	Power	Type	Cost	Power	Interconnect	Bandwidth
Splitwise-AA	DGX-A100	1×	1×	DGX-A100	1×	1×	1×	
Splitwise-HH	DGX-H100	2.35×	1.75×	DGX-H100	2.5×	1.75×	2×	
Splitwise-HHcap	DGX-H100	2.35×	1.75×	DGX-H100	2.5×	1.23×	2×	
Splitwise-HA	DGX-H100	2.35×	1.75×	DGX-A100	1×	1×	1×	

表 V: 评估的分割设计全部归一化为 DGX-A100

云服务提供商 (CSP) 和用户。成本优化对于 CSP 和用户具有不同的重要性级别。对于 CSP 来说, 如果集群的功率和空间要求有所提高, 则相同吞吐量的更高成本可能是可以接受的。然而, 对于最终用户来说, 相同吞吐量下更高的成本通常是不可接受的。最后, 功耗优化对 CSP 很有吸引力, 因为它可以在同一数据中心部署更多 GPU [62]、[63], 但对用户来说可能并不那么重要。在我们的研究中, 我们只考虑配置的功率, 而不考虑动态功率利用率。

E. Practical Considerations

精度影响。Splitwise 不会影响准确性, 因为它使用无损 KV 缓存传输并且不添加任何随机化。它使用与单机相同的参数和状态执行推理。

可扩展性。由于 LLM 请求比典型的 ML 请求 [37]、[38] 长得多, 因此对于相似的集群大小, 它们产生的调度开销较低。然而, CLS 可能成为大型集群的可扩展性瓶颈。先前关于分区或复制调度工作的见解可以帮助提高可扩展性 [27]、[61]、[72], 并且与 Splitwise 正交。

可靠性和容错性。如果提示或令牌机失败, Splitwise 只需从头开始重新启动请求, 类似于当今的 LLM 服务系统 [44]、[51]。或者, Splitwise 可以将即时计算后生成的 KV 缓存检查点到内存数据库中。为了恢复, Splitwise 可以使用此缓存来跳过提示重新计算, 并立即开始令牌阶段。KV 缓存还可以在令牌阶段定期检查点。设计安全高效的故障恢复超出了我们论文的范围。

五、方法论

A. Experimental setup

为了在真实硬件上评估我们的建议, 我们在 vLLM 之上实现了 Splitwise 的 KV 缓存传输机制 [51]。我们的实现是开源的 [1]。我们在 Microsoft Azure 上的两台 DGX-A100 和两台 DGX-H100 虚拟机 (VM) 上运行此修改后的 vLLM, 其规格来自表 I。这些是用于收集第 III 节中的特征数据的 VM。这些机器通过 InfiniBand 连接, DGX-H100 具有双倍带宽 (i.e., 400 Gbps)。

由于普通 vLLM 仅支持带有令牌抢占的连续批处理, 这可能会导致更高的 TBT, 因此我们实现了最先进的混合连续批处理 [81], 如图 2(c) 中先前讨论的那样。

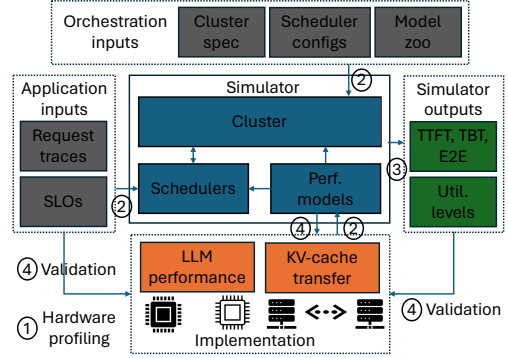


图 13: Splitwise 模拟器的设计概述。

我们实施的 Splitwise 技术为机器分配了提示角色或令牌角色。当提示机生成第一个令牌时, 它使用第 IV-C 节中描述的技术将 KV 缓存传输到令牌机。我们使用 MSCCL++ [11] (一个优化的 GPU 驱动的通信库) 来实现简单且分层的 KV 缓存传输。

在我们的实现中, 提示机使用 MSCCL++ 的零复制单侧放置原语, 一旦准备好就通过 InfiniBand 发送 KV 缓存数据, 而不需要令牌机发出任何接收指令。一旦我们为所有层发出了 put, 提示机就会发出令牌机等待的信号量。借助信号量完成的同步使用与发送 KV 缓存数据相同的 InfiniBand 连接。当处理一批提示时, 每个请求都被分配一个不同的信号量, 因为它可能被路由到不同的令牌机。我们在 vLLM 中逐块传送 KV 缓存。为了最大限度地减少传输次数, 我们还考虑了 KV 块的连续性, 只要它们使用相同的信号量即可。

B. Simulator setup

我们构建了一个模拟器来探索集群设计并大规模评估 Splitwise。模拟器代码是开源的 [20]。

图 13 显示了我们的模拟器的设计。该模拟器是事件驱动的, 忠实地模拟了 Splitwise 机器池、调度程序、机器级内存和队列以及 KV 缓存传输。我们首先在具有各种输入/输出大小的目标硬件上分析 LLM 1。基于特征配置文件, 我们构建了性能模型。模拟器将请求跟踪、SLO、性能模型以及集群和调度程序 2 的配置作为输入。为了我们的

	P50	P90	P99
TTFT	2×	3×	6×
TBT	1.25×	1.5×	5×
E2E	1.25×	1.5×	5×

TABLE VI: SLO expressed as slowdown compared to a request running on DGX-A100 under no contention.

evaluation, we use the prompt and token size distributions from the production traces in Section III. We tune the Poisson arrival rate to increase and decrease the load (requests per second) for cluster sizing. The simulator provides the achieved metrics per request (TTFT, TBT, E2E), and the machine utilization levels (3). We cross-validated the performance model with hardware experiments to ensure accuracy; we also validated the simulator end-to-end using production load with over 50K iterations to ensure fidelity (4).

Performance model. We build a piece-wise linear performance model using performance profiles at various batch sizes, input sizes, output sizes, in the required parallelism configuration on A100 and H100 machines from Section III. We validate that our performance model has high accuracy; it incurs a mean absolute percentage error (MAPE) of less than 3% when evaluated with a 80:20 train:test dataset split.

Communication model. In our evaluation, KV-cache transfers cause inter-machine communication, whereas tensor parallelism only causes intra-machine communication. We model inter-machine communication overheads by benchmarking our KV-cache transfer implementation over Infiniband in Section VI-A.

SLOs. To determine the maximum throughput that can be supported by a given cluster design, we use P50, P90, and P99 SLOs for TTFT, TBT, and E2E latency metrics. Table VI shows our SLO definition using DGX-A100 as a reference. We require all nine SLOs to be met. SLOs on TTFT are slightly looser, since it has a much smaller impact on the E2E latency.

Baselines. We compare our Splitwise designs against Baseline-A100 and Baseline-H100. The clusters in these baselines consist of just DGX-A100s and DGX-H100s, respectively. Both baselines use the same mixed continuous batching that Splitwise uses for mixed pool machines (described in Section IV-A).

VI. EVALUATION

A. Experimental results

KV-cache transfer latency. We first measure the latency to transfer the KV-cache as the prompt size grows. Figure 14 shows the visible transfer latency on both A100 and H100 setups with the naive and optimized transfer design as discussed in Figure 11. Compared to the prompt computation time, the overhead is minimal ($< 7\%$). The time for serialized transfers linearly increases with the prompt size since the size of the KV-cache also increases. The optimized per-layer transfer, on the other hand, hides much of the latency. For these transfers, we see a constant non-overlapped transfer time of around 8ms for the A100 and around 5ms for the H100 setup. The H100

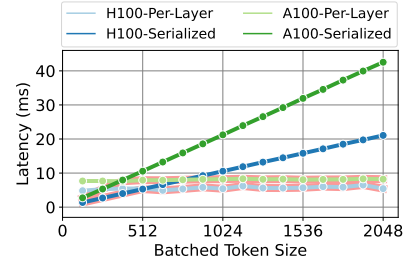


Fig. 14: Overhead of the KV-cache transfer as the prompt size increases on A100s and H100s.

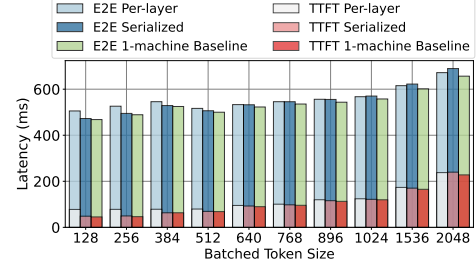


Fig. 15: Overhead of KV cache transfer on TTFT, E2E latency for coding trace for A100 and H100.

setup has double the bandwidth of the A100 setup (*i.e.*, 200 vs 400 Gbps), and the impact of this can be clearly seen with transfers in the H100 setup happening about twice as fast as those in the A100 setup.

As discussed in Section IV-C, for small prompt sizes (< 512 in H100), Splitwise uses the serialized KV-cache transfer and for larger prompts, it uses per-layer transfers.

End-to-end impact. Next, we run the coding trace on the 2-machine Splitwise setups without batching, and compare the observed latency metrics to a 1-machine baseline setup with no batching. Figure 15 shows our results. The latency impact of serially transferring the KV-cache grows up to 3% of the E2E with large prompts. However, Splitwise only incurs 0.8% of E2E. In a user-facing inference, the only visible impact of KV-cache transfer overhead is the latency for the second token. Splitwise adds a 16.5% latency to the second token, as compared to the 64% overhead from a serialized transfer. Overall, the transfer impact in Splitwise is hardly perceivable even in a user-facing inference.

B. Iso-power throughput-optimized clusters

Cluster provisioning. We provision clusters using the methodology described in Section IV-D. We target a specific workload (*e.g.*, conversation) at a peak load with the same power (*i.e.*, iso-power) for each cluster design. For the baseline, we use the power for 40 DGX-H100 machines as our target peak power. For the A100 baseline, we can fit 70 DGX-A100 machines under the same power budget. We denote these two designs as 40P/T and 70P/T respectively, since they both use mixed batching in all machines.

	P50	P90	P99
TTFT	2×	3×	6×
TBT	1.25×	1.5×	5×
E2E	1.25×	1.5×	5×

表 VI: SLO 表示为与无争用情况下在 DGX-A100 上运行的请求相比的减慢。

评估时，我们使用第三节中生产轨迹中的提示和令牌大小分布。我们调整泊松到达率来增加和减少集群大小的负载（每秒请求数）。模拟器提供每个请求所实现的指标（TTFT、TBT、E2E）以及机器利用率水平 3。我们通过硬件实验对性能模型进行交叉验证，以确保准确性；我们还使用超过 50K 次迭代的生产负载对模拟器进行了端到端验证，以确保保真度 4。

性能模型。我们使用第 III 节中的 A100 和 H100 机器上所需的并行配置中的各种批量大小、输入大小、输出大小的性能配置文件构建了分段线性性能模型。我们验证我们的性能模型具有很高的准确性；当使用 80:20 的训练：测试数据集分割进行评估时，它的平均绝对百分比误差 (MAPE) 小于 3%。

通讯模型。在我们的评估中，KV 缓存传输会导致机器间通信，而张量并行仅导致机器内通信。我们通过在第 VI-A 节中对 Infiniband 上的 KV 缓存传输实现进行基准测试来对机器间通信开销进行建模。

SLO。为了确定给定集群设计可以支持的最大吞吐量，我们使用 P50、P90 和 P99 SLO 来衡量 TTFT、TBT 和 E2E 延迟指标。表 VI 显示了我们使用 DGX-A100 作为参考的 SLO 定义。我们要求满足所有九个 SLO。TTFT 上的 SLO 稍微宽松一些，因为它对 E2E 延迟的影响要小得多。

基线。我们将我们的 Splitwise 设计与 Baseline-A100 和 Baseline-H100 进行比较。这些基线中的集群分别仅由 DGX-A100 和 DGX-H100 组成。两个基线都使用与 Splitwise 用于混合池机器的相同混合连续批处理（如第 IV-A 节所述）。

六. 评估

A. Experimental results

KV-缓存传输延迟。我们首先测量随着提示大小的增长传输 KV 缓存的延迟。图 14 显示了 A100 和 H100 设置上的可见传输延迟，采用图 11 中讨论的简单且优化的传输设计。与即时计算时间相比，开销很小 (< 7%)。序列化传输的时间随着提示大小线性增加，因为 KV 缓存的大小也会增加。另一方面，优化的每层传输隐藏了大部分延迟。对于这些传输，我们看到 A100 设置的恒定非重叠传输时间约为 8 毫秒，H100 设置约为 5 毫秒。H100

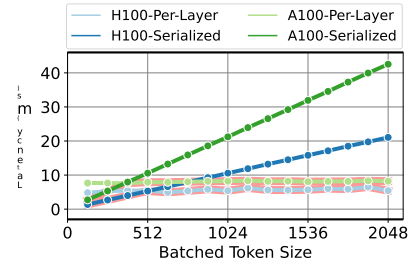


图 14: A100 和 H100 上提示大小增加时 KV 缓存传输的开销。

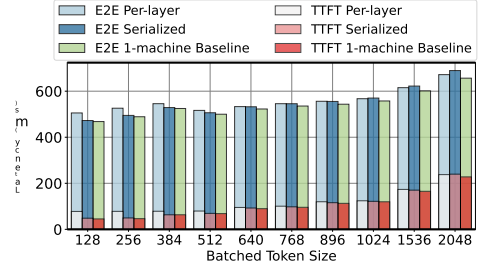


图 15: TTFT 上 KV 缓存传输的开销，A100 和 H100 编码跟踪的 E2E 延迟。

设置的带宽是 A100 设置的两倍 (*i.e.*, 200 与 400 Gbps)，并且可以清楚地看到这一影响，H100 设置中的传输速度大约是 A100 设置中传输速度的两倍。

正如第 IV-C 节中所讨论的，对于较小的提示大小（H100 中的 < 512），Splitwise 使用序列化 KV 缓存传输，对于较大的提示，它使用每层传输。

端到端的影响。接下来，我们在没有批处理的 2 台机器 Splitwise 设置上运行编码跟踪，并将观察到的延迟指标与没有批处理的 1 台机器基线设置进行比较。图 15 显示了我们的结果。串行传输 KV 缓存的延迟影响会在较大的提示下增长到 E2E 的 3%。然而，Splitwise 只产生 E2E 的 0.8%。在面向用户的推理中，KV 缓存传输开销的唯一可见影响是第二个令牌的延迟。Splitwise 为第二个令牌增加了 16.5% 的延迟，而串行传输的开销为 64%。总体而言，即使在面向用户的推理中，Splitwise 的传输影响也很难被察觉。

B. Iso-power throughput-optimized clusters

集群配置。我们使用第 IV-D 节中描述的方法来配置集群。对于每个集群设计，我们以相同功率 (*i.e.*, iso-power) 的峰值负载时的特定工作负载 (*e.g.*, 对话) 为目标。对于基线，我们使用 40 台 DGX-H100 机器的功率作为我们的目标峰值功率。对于 A100 基准，我们可以在相同的功率预算下安装 70 台 DGX-A100 机器。我们将这两种设计分别表示为 40P/T 和 70P/T，因为它们在所有机器中都使用混合配料。

For Splitwise cluster designs under the coding trace, Splitwise-AA provisions 55 prompt machines and 15 for the token pool, denoted as (55P, 15T). Note that like Baseline-A100, Splitwise-AA also provisions 75% more machines than Baseline-H100. The legends in Figure 16 show the different provisioning choices under coding and conversation workloads. Request size distributions reflect in the machine pool sizing. For example, we provision more prompt machines under Splitwise-HH (35P, 5T) for the coding trace, while we provision more token machines (25P, 15T) for the conversation trace.

Latency and throughput. Figure 16 shows a deep dive into all the latency metrics at different input load for each cluster design with the same power (*i.e.*, iso-power). For the coding trace (Figure 16a), Splitwise-HH, Splitwise-HHcap, and Splitwise-AA all perform better than Baseline-H100. As the load increases, Baseline-H100 suffers from high TBT due to mixed batching with large prompt sizes. Although Splitwise-AA can support higher throughput, its TTFT is consistently higher than most designs. Splitwise-HA clearly bridges the gap by providing low TTFT and E2E at high throughput. The mixed machine pool in Splitwise becomes useful at higher loads to use all the available hardware without fragmentation. This benefit can be seen clearly in the P50 TBT chart for Splitwise-HA, where after 90 RPS, H100 machines jump into the mixed machine pool and help reduce TBT.

For the conversation trace (Figure 16b), Splitwise-HHcap clearly does better on all fronts, including latency. This is because its token generation phases typically run for much longer than in the coding trace, which is beneficial for the token machines.

Impact on batched tokens. Figure 17 shows the cumulative distribution of time spent processing a varying number of batched active tokens in an iso-power throughput-optimized cluster. The distributions are collected by running the conversation trace at low (70 RPS) and high (130 RPS) loads.

At low load, all 40 Baseline-H100 machines spend 70% of the time running ≤ 15 tokens, and the rest running mixed batches with large prompts, which affects TBT and E2E. The 35 Splitwise-HH prompt machines are mostly idle, and when active, run much larger batches of tokens. The 15 Splitwise-HH token machines also do a better job at batching. Overall, Splitwise machines have better batching and latency at 70 RPS. At high load, since the mixed pool is utilized more, the batch sizes start looking similar across prompt and token machines.

Summary plot. Figure 18a summarizes the results across all cluster metrics for iso-power throughput-optimized designs for the conversation trace. We use Baseline-A100 as the baseline. Compared to Baseline-A100, Splitwise-AA delivers $2.15\times$ more throughput at the same power and cost. Splitwise-HA delivers $1.18\times$ more throughput at 10% lower cost and the same power.

C. Other cluster optimizations

We have described *iso-power throughput-optimized clusters* in detail. For the rest of the cluster optimization evaluation, we only discuss the summary plots.

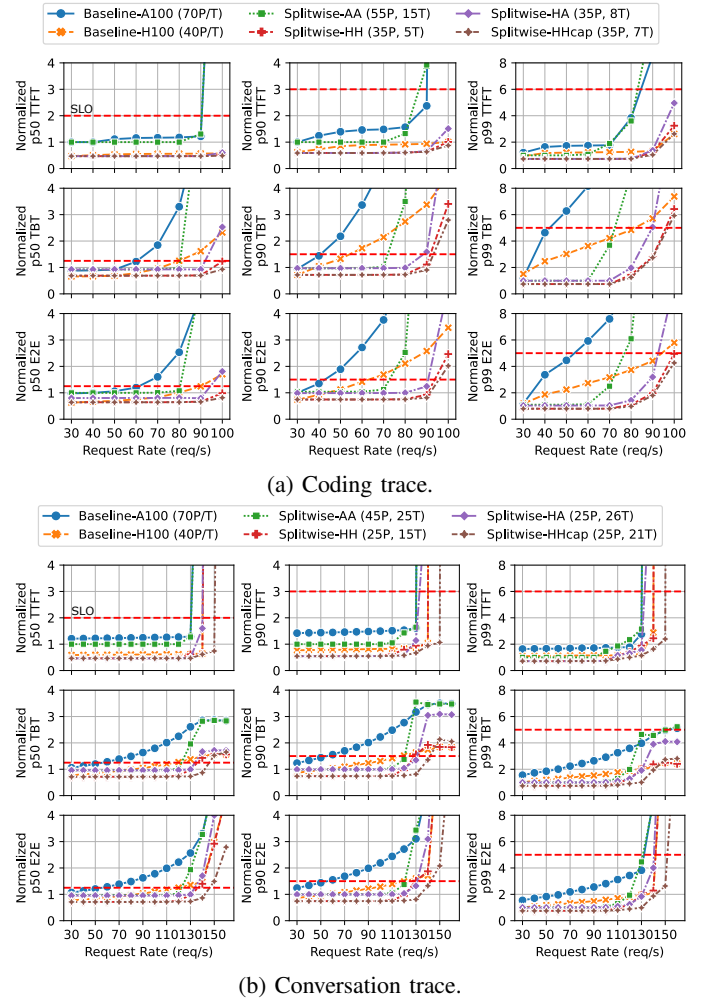


Fig. 16: Latency metrics across input loads for iso-power throughput optimized clusters. Dashed red lines indicate SLO.

Iso-cost throughput-optimized. Figure 18b shows the summary plot for iso-cost clusters, with their space, throughput, and power requirements. We find that Splitwise-AA provides the best throughput for the same cost, namely $1.4\times$ more throughput than Baseline-H100, running at 25% more power, and $2\times$ the space. This is an interesting operational point for most customers who may not care about power and space, instead preferring the 40% higher throughput using older, more easily available GPUs. In contrast, the preferable choice for the CSP is less clear.

Iso-throughput power-optimized. Figure 19a shows cluster designs that yield same throughput at the least power. Splitwise-HHcap can achieve the same throughput as Baseline-H100 at 25% lower power at the same cost and space. This can be a clear win for the CSPs.

Iso-throughput cost-optimized. Figure 19b shows the cost-optimized versions of the iso-throughput design. Note that there are no changes to any of the homogeneous designs between Figures 19a and 19b. This is because the prompt and token machines have the same cost and power. However, Splitwise-

对于编码跟踪下的Splitwise集群设计，Splitwise-AA提供了55个提示机和15个用于令牌池，表示为(55P, 15P)。请注意，与Baseline-A100一样，Splitwise-AA也比Baseline-H100多提供75%的机器。图16中的图例显示了编码和对话工作负载下的不同配置选择。请求大小分布反映在机器池大小中。例如，我们在Splitwise-HH(35P, 5T)下为编码跟踪提供更多提示机，同时对对话跟踪提供更多令牌机(25P, 15T)。

延迟和吞吐量。图16深入研究了具有相同功率(*i.e.*, iso-power)的每个集群设计在不同输入负载下的所有延迟指标。对于编码轨迹(图16a)，Splitwise-HH、Splitwise-HHcap和Splitwise-AA的性能均优于Baseline-H100。随着负载的增加，Baseline-H100由于混合批处理和大提示尺寸而面临高TBT。尽管Splitwise-AA可以支持更高的吞吐量，但其TTFT始终高于大多数设计。Splitwise-HA通过在高吞吐量下提供低TTFT和E2E显然弥补了这一差距。Splitwise中的混合机器池在较高负载下变得非常有用，可以使用所有可用硬件而不会产生碎片。这一优势可以在Splitwise-HA的P50 TBT图表中清楚地看到，其中在90 RPS后，H100机器跳入混合机器池并有助于减少TBT。

对于对话跟踪(图16b)，Splitwise-HHcap显然在所有方面都做得更好，包括延迟。这是因为其令牌生成阶段通常比编码跟踪运行的时间长得多，这对令牌机有利。

对批量代币的影响。图17显示了在等功率吞吐量优化集群中处理不同数量的批量活动令牌所花费的时间的累积分布。通过在低负载(70 RPS)和高负载(130 RPS)下运行对话跟踪来收集分布。

在低负载下，所有40台Baseline-H100机器有70%的时间运行 ≤ 15 个令牌，其余的则运行具有较大提示的混合批次，这会影TBT和E2E。35台Splitwise-HH提示机器大部分处于闲置状态，当处于活动状态时，会运行更大批量的令牌。15个Splitwise-HH令牌机在批处理方面也做得更好。总体而言，Splitwise机器在70 RPS时具有更好的批处理和延迟。在高负载下，由于混合池的利用率更高，因此提示机和令牌机的批量大小开始看起来相似。

概要情节。图18a总结了对话跟踪的等功率吞吐量优化设计的所有集群指标的结果。我们使用Baseline-A100作为基线。与Baseline-A100相比，Splitwise-AA在相同的功耗和成本下可提供2.15 $\times$ 更多的吞吐量。Splitwise-HA在相同功率的情况下以降低10%的成本提供1.18 $\times$ 的吞吐量。

C. Other cluster optimizations

我们已经详细描述了*iso-power throughput-optimized clusters*。对于集群优化评估的其余部分，我们仅讨论摘要图。

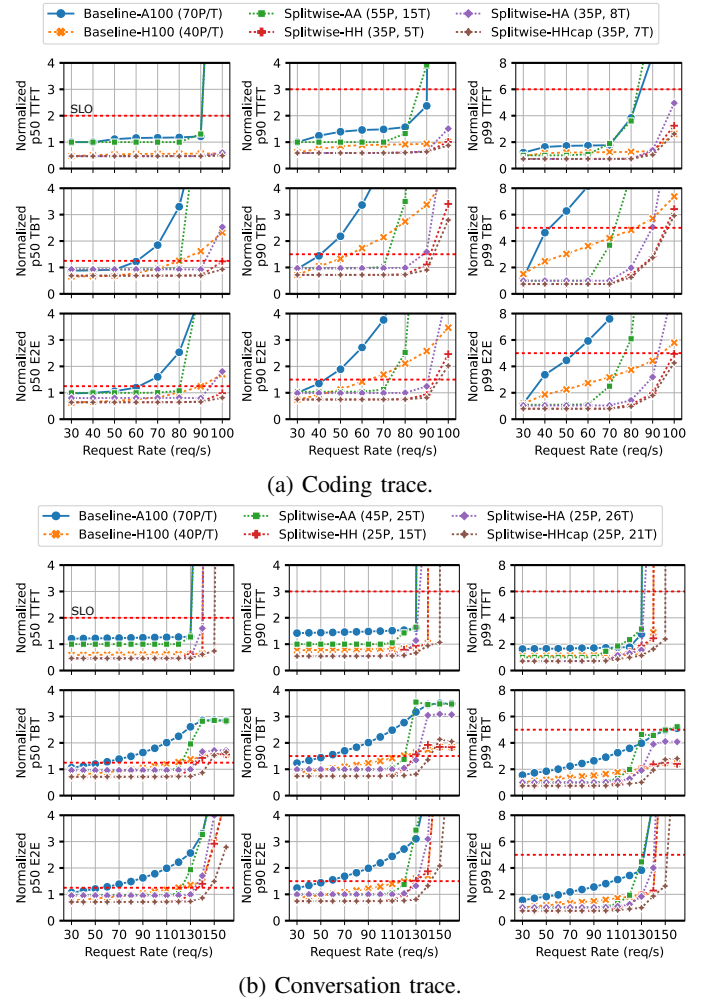


图16: 等功率吞吐量优化集群的输入负载的延迟指标。红色虚线表示SLO。

等成本吞吐量优化。图18b显示了等成本集群的汇总图，以及它们的空间、吞吐量和功率要求。我们发现，Splitwise-AA在相同成本下提供了最佳吞吐量，即比Baseline-H100高1.4 $\times$ 吞吐量，运行功耗高25%，空间增加2 $\times$ 。对于大多数可能不关心功耗和空间，而是更喜欢使用旧的、更容易使用的GPU提高40%吞吐量的客户来说，这是一个有趣的操作点。相比之下，CSP的最佳选择则不太明确。

等吞吐量功率优化。图19a显示了以最低功耗产生相同吞吐量的集群设计。Splitwise-HHcap可以在成本和空间相同的情况下，以降低25%的功耗实现与Baseline-H100相同的吞吐量。这对于CSP来说是明显的胜利。

等吞吐量成本优化。图19b显示了等吞吐量设计的成本优化版本。请注意，图19a和19b之间的同质设计没有任何变化。这是因为提示机和令牌机具有相同的成本和功率。然而，分裂——

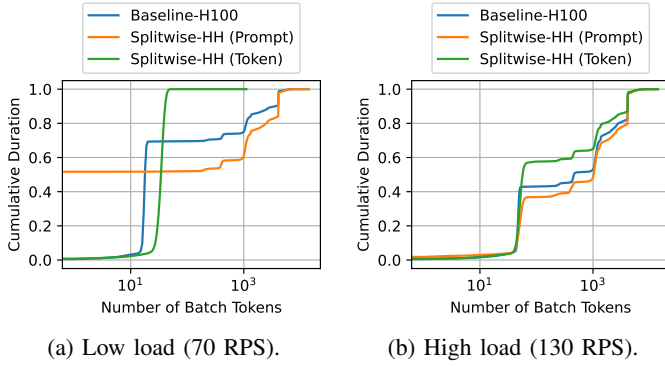


Fig. 17: Cumulative distribution of time spent at various batched token sizes for iso-power throughput-optimized design.

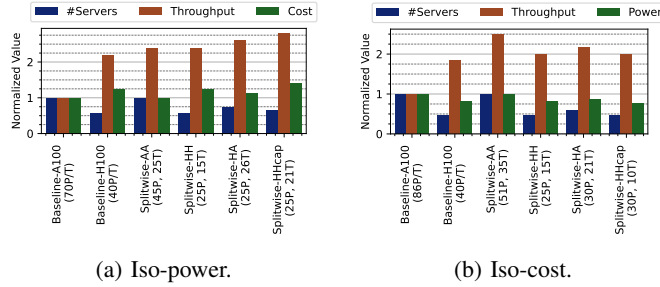


Fig. 18: Summary of throughput-optimized cluster designs.

HA and Splitwise-HHcap arrive at slightly different results with the cost and power optimizations. Figure 19b shows that with Splitwise-AA, customers can achieve the same throughput as Baseline-H100 at 25% lower cost.

D. Impact of workload changes

So far, we have tested a trace and a model on clusters optimized for a specific workload pattern and model. To test the Splitwise' robustness, we now run conversation trace on a cluster meant for coding service, and Llama-70B on a cluster meant for BLOOM-176B. Figure 20 shows these results for iso-power throughput-optimized clusters.

Changing workload trace. Compared to Figure 16b, we find that in Figure 20a, the Baseline clusters are similarly sized and incur no throughput or latency impact. Splitwise-AA and Splitwise-HH with the mixed pool morph well to meet the requirements of the new workload, and they see no throughput or latency impact. Since Splitwise-HA and Splitwise-HHcap have different types of machines in the prompt and token pools, they experience a throughput setback of 7% from the respective cluster optimized designs for conversation trace. Note that all the Splitwise designs still perform much better than any of the Baseline designs.

Changing model. Figure 20b shows that Llama-70B can support much higher throughput in the same cluster design than BLOOM-176B, given its fewer parameters (Table III). All the Splitwise designs out-perform both the Baseline designs at higher load. Furthermore, Splitwise-HH and Splitwise-HHcap consistently achieve the best latency, even as the load increases.

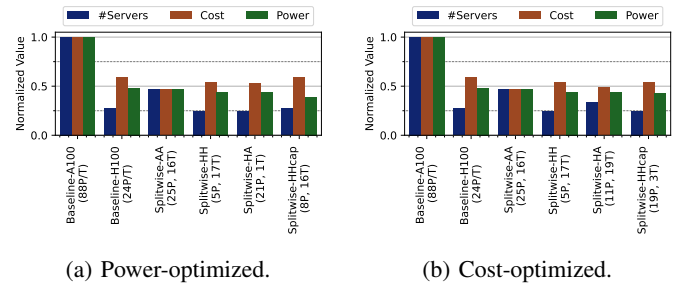


Fig. 19: Summary of iso-throughput cluster designs.

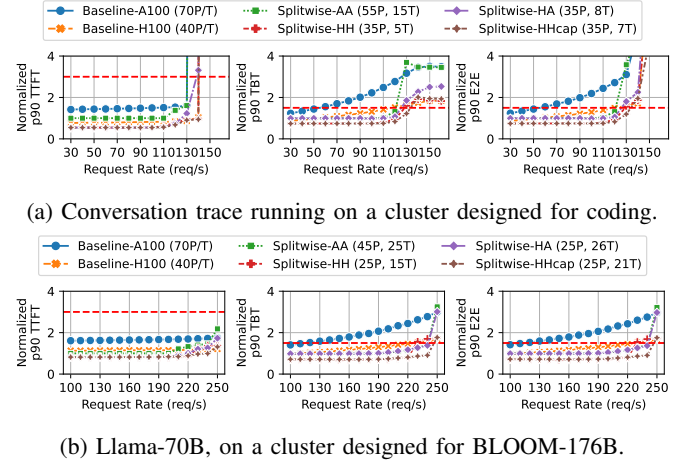


Fig. 20: Latency impact of running a workload on a cluster designed for another workload. Dashed red lines indicate SLO.

Summary. Based on these two experiments, we conclude that Splitwise can morph according to the requirements of the workload using its smart scheduling, and it is robust to changes in the LLMs, request load, and token distributions.

E. Cluster design for batch job

We design various clusters with Splitwise under strict latency SLOs, even when we are optimizing for throughput. This is unnecessary for batch jobs, which can be stressed to high load for a high token generation throughput. We find that upon stressing our iso-power throughput-optimized clusters, Baseline-A100 and Splitwise-AA have the best throughput per cost at 0.89 RPS/\$. At high load, Splitwise devolves into the iso-count Baseline, since it starts mixed batching with all the machines in the mixed pool. The same holds true for Splitwise-HH and Baseline-H100, which achieve 0.75 RPS/\$.

VII. DISCUSSION

Extensibility to new models. Despite the plethora of model sizes from 2B parameters [47], [84] to 176B parameters [69] or more [18], all modern transformer-based generative LLMs have the distinct prompt processing and token generation phases. Similarly, even modifications and flavors like Mixture-of-Experts (MoEs) have these phases. Since Splitwise is built solely by exploiting these phases, it is applicable to all of the

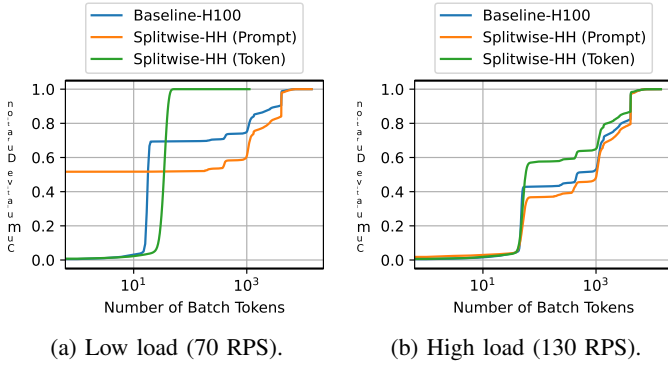


图 17: 等功率吞吐量优化设计中不同批量令牌大小所花费时间的累积分布。

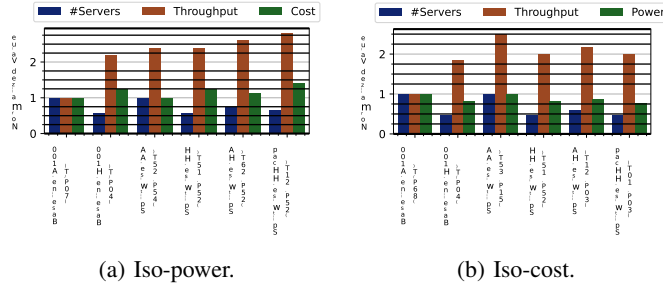


图 18: 吞吐量优化集群设计总结。

HA 和 Splitwise-HHcap 在成本和功耗优化方面得到的结果略有不同。图 19b 显示, 利用 Splitwise-AA, 客户可以降低 25% 的成本实现与 Baseline-H100 相同的吞吐量。

D. Impact of workload changes

到目前为止, 我们已经在针对特定工作负载模式和模型优化的集群上测试了跟踪和模型。为了测试 Splitwise 的稳健性, 我们现在在用于编码服务的集群上运行对话跟踪, 并在用于 BLOOM-176B 的集群上运行 Llama-70B。图 20 显示了等功率吞吐量优化集群的这些结果。

更改工作负载跟踪。与图 16b 相比, 我们发现图 20a 中的基线集群大小相似, 并且不会产生吞吐量或延迟影响。具有混合池的 Splitwise-AA 和 Splitwise-HH 可以很好地满足新工作负载的要求, 并且它们没有看到吞吐量或延迟影响。由于 Splitwise-HA 和 Splitwise-HHcap 在提示池和令牌池中具有不同类型的机器, 因此它们在对话跟踪的各自集群优化设计中遇到了 7% 的吞吐量下降。请注意, 所有 Splitwise 设计的性能仍然比任何 Baseline 设计好得多。

改变模型。图 20b 显示, 由于 Llama-70B 的参数较少 (表 III), 因此在相同的集群设计中, Llama-70B 可以支持比 BLOOM-176B 高得多的吞吐量。所有 Splitwise 设计在较高负载下的性能均优于 Baseline 设计。此外, 即使负载增加, Splitwise-HH 和 Splitwise-HHcap 也始终能实现最佳延迟。

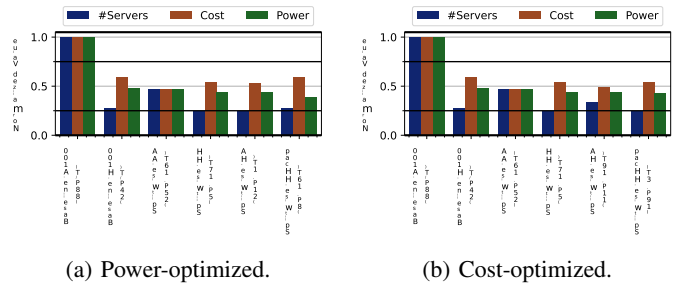


图 19: 等吞吐量集群设计总结。

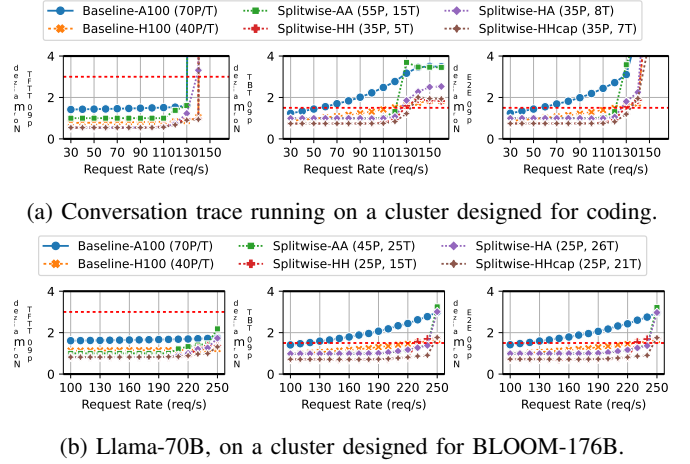


图 20: 在专为另一个工作负载设计的集群上运行一个工作负载的延迟影响。红色虚线表示 SLO。

概括。基于这两个实验, 我们得出结论, Splitwise 可以使用其智能调度根据工作负载的要求进行变化, 并且对于 LLM、请求负载和令牌分布的变化具有鲁棒性。

E. Cluster design for batch job

即使我们正在优化吞吐量, 我们也可以在严格的延迟 SLO 下使用 Splitwise 设计各种集群。这对于批处理作业来说是不必要的, 批处理作业可能会承受高负载以获得高令牌生成吞吐量。我们发现, 在强调我们的等功率吞吐量优化集群时, Baseline-A100 和 Splitwise-AA 的单位成本吞吐量最佳, 为 0.89 RPS/\$。在高负载下, Splitwise 会转变成 iso-count 基线, 因为它开始与混合池中的所有计算机进行混合批处理。Splitwise-HH 和 Baseline-H100 也是如此, 它们达到了 0.75 RPS/\$。

七. 讨论

可扩展至新模型。尽管模型大小从 2B 参数 [47]、[84] 到 176B 参数 [69] 或更多 [18], 但所有现代基于 Transformer 的生成 LLM 都具有独特的提示处理和令牌生成阶段。同样, 即使是像 Mixture-of-Experts (MoE) 这样的修改和风格也有这些阶段。由于 Splitwise 是仅通过利用这些阶段来构建的, 因此它适用于所有

current and upcoming LLMs, as long as the auto-regressive nature of the workload requires these two phases. Note that as shown in [Section VI-D](#), clusters provisioned with Splitwise for one model can also efficiently serve other models.

Alternative compute hardware. In this work, we use NVIDIA H100 and A100 GPUs since they are commonly used for LLM inference in datacenters today [17]. Smaller datacenter GPUs like NVIDIA T4 lack enough memory to run modern LLMs efficiently. In general, our methodology is applicable to any hardware (including CPUs, FPGAs, ASICs [33]) that aligns with the computational requirements of prompt and token phases. Our characterization suggests that prompt phases need high compute capability and memory bandwidth with low memory capacity, whereas token phases need moderate compute capability with high memory capacity and bandwidth. Thus, GPUs like AMD MI-250 [2] and CPUs like Intel Sapphire-Rapids (with HBM) [9] could be effective token machines. Since we do not have access to such hardware and/or optimized LLM implementations, we leave this to future work.

Interconnect between prompt and token machines. In this work, we assume Infiniband connection between the prompt and token machines in all the designs (albeit, lower bandwidth when A100s were involved). Although this is common for all homogenous machines, Splitwise-HA is not readily available with an Infiniband connection between H100s and A100s, even though technically feasible. The alternative could be HPC clouds, with Infiniband connections through the CPU [3], or Ethernet, using RoCE [58]. Given our optimized KV-cache transfer that helps reduce critical latency, an interconnect with $10\times$ lower bandwidth would likely still be beneficial. To further reduce our bandwidth utilization, we could also compress the KV-cache before transferring it across the network [55].

Heterogeneous prompt/token machines. Although Splitwise is robust to varied models and input traces, we recognize that fragmenting a data center with different types of GPUs (*e.g.*, Splitwise-HA) may bring its own challenges for the CSP.

Conversation back and forth. Chat APIs for LLMs today require the user to send the complete context of the conversation so far [18]. However, in the future, services may have enough GPU capacity to cache the context and avoid recomputation. This could sway the memory utilization pattern of the prompt phase from our characterization. Furthermore, it may require transferring the KV-cache back to a prompt machine to be ready for the next conversation request.

VIII. RELATED WORK

Heterogeneous scheduling and dataflow systems. Prior work has studied heterogeneous scheduling for a variety of interactive services [65], [68], [83]. These works exploit hardware heterogeneity to strike a balance between different objectives such as cost, energy, and performance. However, they run the entire workload on the same machine. Research on heterogeneous multiprocessor CPU scheduling attempts to match workload heterogeneity to hardware heterogeneity [29], [40], [41], [50], [76], [80]. These works use profiling or

online monitoring with metrics like request length or hardware performance counters to identify workload phases and allocate them appropriately on heterogeneous processors. However, they do not consider the complexities with batching. Distributed dataflow systems orchestrate large-scale computational graphs and aim to provide general-purpose programmability [34], [46], [75], [82]. LLM inference under Splitwise can be viewed as a static computational graph with two stages, so it could be implemented using distributed frameworks that provide efficient GPU abstractions [59]. Splitwise differs from these works since it uses a specialized two-phase design for generative LLM inference and leverages phase-aware resource management with efficient batching.

Model serving systems. LLM inference serving is a rapidly developing field, with several recent works optimizing batching [23], [25], [51], [53], [81], scheduling [22], [42], [51], [66], [73], [79], and memory usage [32], [35], [51], [74]. Prior work has also proposed using CPUs and lower compute capability devices for LLM serving [8], [12]. These approaches use the same machine for both prompt and token phase. With Splitwise, they could improve throughput and latency by splitting phases.

Prior work on video and ML serving focuses on scheduling model chains with data dependencies under latency constraints [24], [31], [43], [49], [68]. Such schedulers rely on model profiling to make efficient allocation decisions and manage requests across machines. Recommendation system inference exhibits compute/memory heterogeneity both within and across models. Prior work exploits such heterogeneity to selectively schedule requests between CPUs and accelerators [38], [52], colocate models with complementary memory usage [30], and partition compute/memory on heterogeneous hardware resources [45], [48]. Similarly, Splitwise exploits the heterogeneity within LLM inference requests. However, it uses different optimizations due to the differences in LLM workload characteristics and requirements.

IX. CONCLUSION

We extensively characterized the prompt computation and token generation phases of LLM inference to draw out differences in their system utilization patterns. Based on our insights, we designed Splitwise to separate these phases onto different machines and enable phase-specific resource management. Using Splitwise, we explored cluster designs optimized for throughput, cost, and power, and showed that they perform well even as workloads change. Splitwise clusters under performance SLOs achieve $1.76\times$ better throughput with 15% lower power at the same cost, or $2.35\times$ better throughput with same the cost and power than existing designs.

ACKNOWLEDGEMENTS

We thank the reviewers for their helpful feedback. We thank Chetan Bansal, Srikant Bhardwaj, Suriya Kalivardhan, Ankur Mallick, Deepak Narayanan, and Amar Phanishayee for insightful discussions. Pratyush Patel was partially supported by NSF CNS-2104548 and a research grant from VMware.

当前和即将到来的**大语言模型**主要工作量的自回归性质需要这两个阶段。请注意，如第 VI-D 节所示，为一个模型配置 Splitwise 的集群也可以有效地为其他模型提供服务。

替代计算硬件。在这项工作中，我们使用 NVIDIA H100 和 A100 GPU，因为它们通常用于当今数据中心的 LLM 推理 [17]。NVIDIA T4 等较小的数据中心 GPU 缺乏足够的内存来有效运行现代 LLM。一般来说，我们的方法适用于任何符合提示和令牌阶段计算要求的硬件（包括 CPU、FPGA、ASIC [33]）。我们的表征表明，提示阶段需要高计算能力和内存带宽以及低内存容量，而令牌阶段需要中等计算能力以及高内存容量和带宽。因此，AMD MI-250 [2] 等 GPU 和 Intel Sapphire-Rapids（带 HBM）[9] 等 CPU 可能是有效的代币机器。由于我们无法访问此类硬件和/或优化的 LLM 实施，因此我们将其留给未来的工作。

提示机和令牌机之间的互连。在这项工作中，我们假设所有设计中的提示机和令牌机之间都有 Infiniband 连接（尽管涉及 A100 时带宽较低）。尽管这对于所有同类机器都很常见，但即使在技术上可行，H100 和 A100 之间的 Infiniband 连接也不容易实现 Splitwise-HA。另一种选择是 HPC 云，通过 CPU [3] 进行 Infiniband 连接，或者使用 RoCE [58] 进行以太网连接。鉴于我们优化的 KV 缓存传输有助于减少关键延迟，带宽降低 10 $\times$ 的互连可能仍然有益。为了进一步降低带宽利用率，我们还可以在通过网络传输 KV 缓存之前对其进行压缩 [55]。

异构提示/令牌机。尽管 Splitwise 对于不同的模型和输入跟踪具有鲁棒性，但我们认识到，使用不同类型的 GPU（*e.g.*、Splitwise-HA）对数据中心进行碎片化可能会给 CSP 带来挑战。

谈话来来回回。如今，**大语言模型**的 API 要求用户发送迄今为止的完整对话上下文 [18]。然而，在未来，服务可能有足够的 GPU 容量来缓存上下文并避免重新计算。这可能会影响我们表征中提示阶段的内存利用模式。此外，可能需要将 KV 缓存传输回提示机，以便为下一个会话请求做好准备。

八. 相关工作

异构调度和数据流系统。先前的工作研究了各种交互服务的异构调度[65]、[68]、[83]。这些工作利用硬件异构性来在成本、能源和性能等不同目标之间取得平衡。然而，他们在同一台机器上运行整个工作负载。异构多处理器CPU调度的研究试图将工作负载异构性与硬件异构性相匹配[29]、[40]、[41]、[50]、[76]、[80]。这些作品使用分析或

使用请求长度或硬件性能计数器等指标进行在线监控，以识别工作负载阶段并在异构处理器上适当地分配它们。然而，他们没有考虑批处理的复杂性。分布式数据流系统编排大规模计算图，旨在提供通用可编程性[34]、[46]、[75]、[82]。Splitwise 下的 LLM 推理可以被视为具有两个阶段的静态计算图，因此可以使用提供高效 GPU 抽象的分布式框架来实现 [59]。Splitwise 与这些工作不同，因为它使用专门的两阶段设计来进行生成 LLM 推理，并利用阶段感知资源管理和高效批处理。

模型服务系统。LLM推理服务是一个快速发展的领域，最近的一些工作优化了批处理[23]、[25]、[51]、[53]、[81]、调度[22]、[42]、[51]、[66]、[73]、[79]和内存使用[32]、[35]、[51]、[74]。先前的工作还建议使用 CPU 和较低计算能力的设备来提供 LLM 服务 [8]、[12]。这些方法在提示阶段和令牌阶段使用同一台机器。借助 Splitwise，他们可以通过分割阶段来提高吞吐量和延迟。

先前关于视频和机器学习服务的工作重点是在延迟约束下具有数据依赖性的调度模型链[24]、[31]、[43]、[49]、[68]。此类调度程序依靠模型分析来做出有效的分配决策并管理跨机器的请求。推荐系统推理在模型内部和模型之间都表现出计算/内存异构性。先前的工作利用这种异构性来选择性地调度 CPU 和加速器之间的请求 [38]、[52]、将模型与互补的内存使用进行共置 [30]，以及在异构硬件资源上对计算/内存进行分区 [45]、[48]。类似地，Splitwise 利用了 LLM 推理请求中的异构性。然而，由于 LLM 工作负载特征和要求的不同，它使用了不同的优化。

九. 结论

我们广泛地描述了 LLM 推理的即时计算和令牌生成阶段，以找出其系统使用模式的差异。根据我们的见解，我们设计了 Splitwise，将这些阶段分离到不同的机器上，并实现特定阶段的资源管理。使用 Splitwise，我们探索了针对吞吐量、成本和功耗进行优化的集群设计，并表明即使工作负载发生变化，它们也能表现良好。与现有设计相比，性能 SLO 下的拆分集群可在相同成本下将吞吐量提高 1.76 $\times$ ，同时功耗降低 15%，或者在成本和功耗相同的情况下将吞吐量提高 2.35 $\times$ 。

致谢

我们感谢审稿人提供的有用反馈。我们感谢 Chetan Bansal、Srikant Bhardwaj、Suriya Kalivardhan、Ankur Mallick、Deepak Narayanan 和 Amar Phanishaye 进行了富有洞察力的讨论。Pratyush Patel 得到了 NSF CNS-2104548 和 VMware 的研究资助的部分支持。

REFERENCES

- [1] Add Splitwise Implementation to vLLM. GitHub. [Online]. Available: <https://github.com/vllm-project/vllm/pull/2809>
- [2] AMD Instinct™ MI250 Accelerator. [Online]. Available: <https://www.amd.com/en/products/server-accelerators/instinct-mi250>
- [3] Azure InfiniBand HPC VMs. [Online]. Available: <https://learn.microsoft.com/en-us/azure/virtual-machines/overview-hb-hc>
- [4] Azure Public Dataset: Azure LLM Inference Trace 2023. GitHub. [Online]. Available: <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md>
- [5] CoreWeave - Specialized Cloud Provider. [Online]. Available: <https://www.coreweave.com>
- [6] Google Assistant with Bard. [Online]. Available: <https://blog.google/products/assistant/google-assistant-bard-generative-ai/>
- [7] HPC Interconnect on CoreWeave Cloud. [Online]. Available: <https://docs.coreweave.com/networking/hpc-interconnect>
- [8] Intel BigDL-LLM. [Online]. Available: <https://github.com/intel-analytics/BigDL>
- [9] Intel Sapphire Rapids with HBM. [Online]. Available: <https://www.anandtech.com/show/17422/intel-showcases-sapphire-rapids-plus-hbm-xeon-performance-isc-2022>
- [10] Microsoft Azure ND A100 v4-series. [Online]. Available: <https://learn.microsoft.com/en-us/azure/virtual-machines/nda100-v4-series>
- [11] MSCCL++: A GPU-driven communication stack for scalable AI applications. [Online]. Available: <https://github.com/microsoft/mscclpp>
- [12] Numenta Inference on CPUs. [Online]. Available: <https://www.servethehome.com/numenta-has-the-secret-to-ai-inference-on-cpus-like-the-intel-xeon-max/>
- [13] NVIDIA Accelerated InfiniBand Solutions. [Online]. Available: <https://www.nvidia.com/en-us/networking/products/infiniband/>
- [14] NVIDIA Chip Shortage. [Online]. Available: <https://www.wired.com/story/nvidia-chip-shortages-leave-ai-startups-scrabbling-for-computing-power/>
- [15] NVIDIA DGX A100: Universal System for AI Infrastructure. [Online]. Available: <https://resources.nvidia.com/en-us-dgx-systems/dgx-ai>
- [16] NVIDIA DGX H100. [Online]. Available: <https://www.nvidia.com/en-us/data-center/dgx-h100/>
- [17] NVIDIA Hopper GPUs Expand Reach as Demand for AI Grows. [Online]. Available: <https://nvidianews.nvidia.com/news/nvidia-hopper-gpus-expand-reach-as-demand-for-ai-grows>
- [18] OpenAI ChatGPT APIs. [Online]. Available: <https://openai.com/blog/introducing-chatgpt-and-whisper-apis>
- [19] Power Availability Stymies Datacenter Growth. [Online]. Available: <https://www.networkworld.com/article/972483/power-availability-stymies-data-center-growth>
- [20] SplitwiseSim: LLM Serving Cluster Simulator. GitHub. [Online]. Available: <https://github.com/Mutinifni/splitwise-sim>
- [21] The New Bing. [Online]. Available: <https://www.microsoft.com/en-us/edge/features/the-new-bing?form=MT00D8>
- [22] TurboMind Inference Server. [Online]. Available: <https://github.com/InternLM/lmdeploy>
- [23] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, and R. Ramjee, "SARATHI: Efficient LLM Inference by Piggybacking Decodes with Chunked Prefixes," *arXiv preprint arXiv:2308.16369*, 2023.
- [24] H. Albahar, S. Dongare, Y. Du, N. Zhao, A. K. Paul, and A. R. Butt, "SchedTune: A heterogeneity-aware GPU Scheduler for Deep Learning," in *CCGrid*, 2022.
- [25] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley, and Y. He, "DeepSpeed-Inference: Enabling Efficient Inference of Transformer Models at Unprecedented Scale," in *SC*, 2022.
- [26] L. A. Barroso, U. Hölzle, and P. Ranganathan, "The Datacenter as a Computer: Designing Warehouse-Scale Machines," *Synthesis Lectures on Computer Architecture*, 2018.
- [27] E. Boutin, J. Ekanayake, W. Lin, B. Shi, J. Zhou, Z. Qian, M. Wu, and L. Zhou, "Apollo: Scalable and Coordinated Scheduling for Cloud-scale Computing," in *OSDI*, 2014.
- [28] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," *arXiv preprint arXiv:2005.14165*, 2020.
- [29] J. Chen and L. K. John, "Efficient Program Scheduling for Heterogeneous Multi-core Processors," in *DAC*, 2009.
- [30] Y. Choi, J. Kim, and M. Rhu, "Hera: A Heterogeneity-Aware Multi-Tenant Inference Server for Personalized Recommendations," *arXiv preprint arXiv:2302.11750*, 2023.
- [31] D. Crankshaw, G.-E. Sela, X. Mo, C. Zumar, I. Stoica, J. Gonzalez, and A. Tumanov, "InferLine: Latency-aware Provisioning and Scaling for Prediction Serving Pipelines," in *SoCC*, 2020.
- [32] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "FlashAttention: Fast and Memory-efficient Exact Attention with IO-Awareness," in *NeurIPS*, 2022.
- [33] David Patterson. Domain Specific Architectures for Deep Neural Networks: Three Generations of Tensor Processing Units (TPUs). Allen School Distinguished Lecture. [Online]. Available: <https://www.youtube.com/watch?v=VCScWh966u4>
- [34] J. Dean and S. Ghemawat, "MapReduce: Simplified Data Processing on Large Clusters," *Communications of the ACM*, 2008.
- [35] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," *arXiv preprint arXiv:2208.07339*, 2022.
- [36] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *NAACL*, 2019.
- [37] A. Gujarati, R. Karimi, S. Alzayat, W. Hao, A. Kaufmann, Y. Vigfusson, and J. Mace, "Serving DNNs like Clockwork: Performance Predictability from the Bottom Up," in *OSDI*, 2020.
- [38] U. Gupta, S. Hsia, V. Saraph, X. Wang, B. Reagen, G.-Y. Wei, H.-H. S. Lee, D. Brooks, and C.-J. Wu, "DeepRecSys: A System for Optimizing End-to-end At-scale Neural Recommendation Inference," in *ISCA*, 2020.
- [39] V. Gupta, M. Harchol Balter, K. Sigman, and W. Whitt, "Analysis of Join-the-Shortest-Queue Routing for Web Server Farms," *Performance Evaluation*, 2007.
- [40] M. E. Haque, Y. H. Eom, Y. He, S. Elnikety, R. Bianchini, and K. S. McKinley, "Few-to-Many: Incremental Parallelism for Reducing Tail Latency in Interactive Services," *ACM SIGPLAN Notices*, 2015.
- [41] M. E. Haque, Y. He, S. Elnikety, T. D. Nguyen, R. Bianchini, and K. S. McKinley, "Exploiting Heterogeneity for Tail Latency and Energy Efficiency," in *MICRO*, 2017.
- [42] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, H. Dong, and Y. Wang, "FlashDecoding++: Faster Large Language Model Inference on GPUs," *arXiv preprint arXiv:2311.01282*, 2023.
- [43] Y. Hu, R. Ghosh, and R. Govindan, "Scrooge: A Cost-effective Deep Learning Inference System," in *SoCC*, 2021.
- [44] Huggingface. Text Generation Inference (TGI). [Online]. Available: <https://github.com/huggingface/text-generation-inference>
- [45] R. Hwang, T. Kim, Y. Kwon, and M. Rhu, "Centaur: A Chiplet-based, Hybrid Sparse-Dense Accelerator for Personalized Recommendations," in *ISCA*, 2020.
- [46] M. Isard, M. Budiu, Y. Yu, A. Birrell, and D. Fetterly, "Dryad: Distributed Data-Parallel Programs from Sequential Building Blocks," in *EuroSys*, 2007.
- [47] M. Javaheripi and S. Bubeck, "Phi-2: The Surprising Power of Small Language Models," *Microsoft Research Blog*, 2023.
- [48] W. Jiang, Z. He, S. Zhang, K. Zeng, L. Feng, J. Zhang, T. Liu, Y. Li, J. Zhou, C. Zhang *et al.*, "FleetRec: Large-scale Recommendation Inference on Hybrid GPU-FPGA Clusters," in *KDD*, 2021.
- [49] R. S. Kannan, L. Subramanian, A. Raju, J. Ahn, J. Mars, and L. Tang, "GrandSLAM: Guaranteeing SLAs for Jobs in Microservices Execution Frameworks," in *EuroSys*, 2019.
- [50] R. Kumar, K. I. Farkas, N. P. Jouppi, P. Ranganathan, and D. M. Tullsen, "Single-ISA Heterogeneous Multi-core Architectures: The Potential for Processor Power Reduction," in *MICRO*, 2003.
- [51] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient Memory Management for Large Language Model Serving with PagedAttention," in *SOSP*, 2023.
- [52] Y. Kwon, Y. Lee, and M. Rhu, "TensorDIMM: A Practical Near-Memory Processing Architecture for Embeddings and Tensor Operations in Deep Learning," in *MICRO*, 2019.
- [53] Z. Li, L. Zheng, Y. Zhong, V. Liu, Y. Sheng, X. Jin, Y. Huang, Z. Chen, H. Zhang, J. E. Gonzalez, and I. Stoica, "AlpaServe: Statistical Multiplexing with Model Parallelism for Deep Learning Serving," in *OSDI*, 2023.

- [1] 将分割实施添加到 vLLM. GitHub. [在线的]. 可用: <https://github.com/vllm-project/vllm/pull/2809> [2] AMD Instinct™ MI250 加速器. [在线的]. 可用: <https://www.amd.com/en/products/server-accelerators/instinct-mi-250> [3] Azure InfiniBand HPC VM. [在线的]. 可用: <https://learn.microsoft.com/en-us/azure/virtual-machines/overview-hb-hc> [4] Azure 公共数据集: Azure LLM 推理跟踪 2023. GitHub. [在线的]. 可用: <https://github.com/Azure/AzurePublicDataset/blob/master/AzureLLMInferenceDataset2023.md> [5] CoreWeave - 专业云提供商. [在线的]. 可用: <https://www.coreweave.com> [6] Google Assistant 与 Bard. [在线的]. 可用: <https://blog.google/products/assistant/google-assistant-bard-generative-ai/> [7] CoreWeave 云上的 HPC 互连. [在线的]. 可用: <https://docs.coreweave.com/networking/hpc-interconnect> [8] 英特尔 BigDL-LLM. [在线的]. 可用: <https://github.com/intel-analytics/BigDL> [9] 采用 HBM 的英特尔 Sapphire Rapids. [在线的]. 可用: <https://www.anandtech.com/show/17422/intel-showcases-sapphire-rapids-plus-hbm-xeon-performance-isc-2022> [10] Microsoft Azure ND A100 v4 系列. [在线的]. 可用: <https://learn.microsoft.com/en-us/azure/virtual-machines/nda100-v4-series> [11] MSCCL++: 用于可扩展 AI 应用程序的 GPU 驱动的通信堆栈. [在线的]. 可用: <https://github.com/microsoft/mscclpp> [12] CPU 上的 Numenta 推理. [在线的]. 可用: <https://www.servethehome.com/numenta-has-the-secret-to-ai-inference-on-cpus-like-the-intel-xeon-max/> [13] NVIDIA 加速 InfiniBand 解决方案. [在线的]. 可用: <https://www.nvidia.com/en-us/networking/products/infiniband/> [14] NVIDIA 芯片短缺. [在线的]. 网址: <https://www.wired.com/story/nvidia-chip-shortages-leave-ai-startups-scrambling-for-computing-power/> [15] NVIDIA DGX A100: 人工智能基础设施通用系统. [在线的]. 可用: <https://resources.nvidia.com/en-us-dgx-systems/dgx-ai> [16] NVIDIA DGX H100. [在线的]. 可用: <https://www.nvidia.com/en-us/data-center/dgx-h100/> [17] NVIDIA Hopper GPU 随着人工智能需求的增长而扩大覆盖范围. [在线的]. 可用: <https://nvidianews.nvidia.com/news/nvidia-hopper-gpus-expand-reach-as-demand-for-ai-grows> [18] OpenAI ChatGPT API. [在线的]. 可用: <https://openai.com/blog/introducing-chatgpt-and-whisper-apis> [19] 电力可用性阻碍数据中心的生长. [在线的]. 可用: <https://www.networkworld.com/article/972483/power-availability-stymies-data-center-growth> [20] SplitwiseSim: LLM 服务集群模拟器. GitHub. [在线的]. 可用: <https://github.com/Mutinifni/splitwise-sim> [21] 新必应. [在线的]. 可用: <https://www.microsoft.com/en-us/edge/features/the-new-bing?form=MT00D8> [22] TurboMind 推理服务器. [在线的]. 可用: <https://github.com/InternLM/lmdeploy> [23] A. Agrawal, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani 和 R. Ramjee, “SARATHI: 使用分块预填充通过 Piggybacking 解码实现高效 LLM 推理”, *arXiv preprint arXiv:2308.16369*, 2023 年. [24] H. Albahar, S. Dongare, Y. Du, N. 赵, A.K. Paul 和 A. R. Butt, “SchedTune: 用于深度学习的异构感知 GPU 调度程序”, *CCGrid*, 2022 年. [25] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Cheng, O. Ruwase, S. Smith, M. Zhu, J. Rasley 和 Y. He, “DeepSpeed-推理: 以前所未有的规模实现变压器模型的高效推理”, *SC*, 2022 年. [26] L. A. Barroso, U. Hözl 和 P. Ranganathan, “作为计算机的数据中心: 设计仓库规模的机器”, *Synthesis Lectures on Computer Architecture*, 2018 年. [27] E. Boutin, J. Ekanayake, W. Lin, B. Shi, J. Zhou, Z. Chen, M. Wu 和 L. Zhou, “Apollo: 云规模计算的可扩展和协调调度”, 载于 *OSDI*, 2014 年. [28] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. 苏茨克维尔, 和 D. Amodei, “语言模型是少样本学习者”, *arXiv preprint arXiv:2005.14165*, 2020. [29] J. Chen 和 L. K. John, “异构多核处理器的高效程序调度”, *DAC*, 2009. [30] Y. Choi, J. Kim 和 M. R. hu, “Hera: 异构感知用于个性化推荐的多租户推理服务器”, *arXiv preprint arXiv:2302.11750*, 2023 年. [31] D. Crankshaw, G.-E. Se la, X. Mo, C. Zumar, I. Stoica, J. Gonzalez 和 A. Tumanov, “InferLine: 预测服务管道的延迟感知配置和扩展”, *SoCC*, 2020 年. [32] T. Dao, D. Fu, S. Ermon, A. Rudra 和 C. Ré, “FlashAttention: 快速且高效的精确注意力机制 IO 意识”, 载于 *NeurIPS*, 2022 年. [33] David Patterson. 深度神经网络的领域特定架构: 三代张量处理单元 (TPU). 艾伦学校杰出讲座. [在线的]. 可用: <https://www.youtube.com/watch?v=VCScWh966u4> [34] J. Dean 和 S. Ghemawat, “MapReduce: 大型集群上的简化数据处理”, *Communications of the ACM*, 2008. [35] T. Dettmers, M. Lewis, Y. Belkada 和 L. Zettlemoyer, “LLM.int8(): 8 位矩阵乘法大规模变形金刚”, *arXiv preprint arXiv:2208.07339*, 2022 年. [36] J. Devlin, M. -W. Chang, K. Lee 和 K. Toutanova, “BERT: 用于语言理解的深度双向变换器的预训练”, *NAACL*, 2019 年. [37] A. Gujarati, R. Karimi, S. Alzayat, W. hao, A. Kaufmann, Y. Vigfusson 和 J. Mace, “为 Clockwork 等 DNN 提供服务: 自下而上的性能可预测性”, 载于 *NAACL. OSDI*, 2020. [38] U. Gupta, S. Hsia, V. Saraph, X. Wang, B. Reagen, G.-Y. 魏, H.-H. S. Lee, D. Brooks 和 C.-J. Wu, “DeepRecSys: 优化端到端大规模神经推荐推理的系统”, *ISCA*, 2020 年. [39] V. Gupta, M. Harchol Balter, K. Sigman 和 W. Whitt, “Web 服务器群加入最短队列路由的分析”, *Performance Evaluation*, 2007 年. [40] M. E. Haque, Y. H. Eom, Y. He, S. Elnikety, R. Bianchini 和 K. S. McKinley, “少对多: 减少交互式服务尾部延迟的增量并行性”, *ACM SIGPLAN Notices*, 2015. [41] M. E. Haque, Y. He, S. Elnikety, T. D. Nguyen, R. Bianchini 和 K. S. McKinley, “利用异构性实现尾部延迟和能源效率”, *MICRO*, 2017 年. [42] K. Hong, G. Dai, J. Xu, Q. Mao, X. Li, J. Liu, K. Chen, H. Dong 和 Y. Wang, “FlashDecoding++: GPU 上更快的大型语言模型推理”, *arXiv preprint arXiv:2311.01282*, 2023 年. [43] Y. Hu, R. Ghosh 和 R. Govindan, “Scrooge: 一种经济有效的深度学习推理系统”, 见 *SoCC*, 2021 年. [44] Huggingface. 文本生成推理 (TGI). [在线的]. 可用: <https://github.com/huggingface/text-generation-inference> [45] R. Hwang, T. Kim, Y. Kwon 和 M. Rhu, “Centaur: 基于 Chiplet 的混合稀疏密集加速器, 用于个性化推荐”, *ISCA*, 2020 年. [46] M. Isard, M. Budiu, Y. Yu, A. Birrell 和 D. Fetterly, “Dryad: 来自顺序构建块的分布式数据并行程序”, *EuroSys*, 2007. [47] M. Javaheripi 和 S. Bubeck, “Phi-2: 小语言模型的惊人力量”, *Microsoft Research Blog*, 2023. [48] W. J. iang, Z. He, S. Zhang, K. Zeng, L. Feng, J. Zhu, T. Liu, Y. Li, J. Zhou, C. Zhang *et al.*, “FleetRec: 混合 GPU-FPGA 集群上的大规模推荐推理”, *KDD*, 2021 年. [49] R. S. Kannan, L. Subramanian, A. Raju, J. Ahn, J. Mars 和 L. Tang, “GrandSLAM: 保证微服务执行中作业的 SLA 框架”, 载于 *EuroSys*, 2019 年. [50] R. Kumar, K. I. Farkas, N. P. Jouppi, P. Ranganathan 和 D. M. Tullsen, “单 ISA 异构多核架构: 降低处理器功耗的潜力”, 载于 *MICRO*, 2003 年. [51] W. Kwon, Z. Li, S. Zhuang, Y. heng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang 和 I. Stoica, “利用 PagedAttention 进行大型语言模型的高效内存管理”, *SOSP*, 2023 年. [52] Y. Kwon, Y. Lee 和 M. Rhu, “TensorDIMM: 一种用于深度学习中嵌入和张量操作的实用近内存处理架构学习”, *MICRO*, 2019 年. [53] Z. Li, L. Cheng, Y. zhong, V. Liu, Y. Shen, X. Jin, Y. Huang, Z. Chen, H. 张, J. E. Gonzalez 和 I. Stoica, “AlpaServe: 用于深度学习服务的模型并行统计复用”, *OSDI*, 2023 年.

- [54] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv preprint arXiv:1907.11692*, 2019.
- [55] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re, and B. Chen, “Deja Vu: Contextual Sparsity for Efficient LLMs at Inference Time,” in *ICML*, 2023.
- [56] Meta. Introducing the AI Research SuperCluster — Meta’s Cutting-Edge AI Supercomputer for AI Research. [Online]. Available: <https://ai.facebook.com/blog/ai-rsc/>
- [57] “Azure OpenAI Service,” Microsoft Azure, 2022. [Online]. Available: <https://azure.microsoft.com/en-us/products/ai-services/openai-service>
- [58] R. Mittal, A. Shpiner, A. Panda, E. Zahavi, A. Krishnamurthy, S. Ratnasamy, and S. Shenker, “Revisiting Network Support for RDMA,” *arXiv preprint arXiv:1806.08159*, 2018.
- [59] P. Moritz, R. Nishihara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan *et al.*, “Ray: A Distributed Framework for Emerging AI Applications,” in *OSDI*, 2018.
- [60] OpenAI. Scaling Kubernetes to 7,500 Nodes. [Online]. Available: <https://openai.com/research/scaling-kubernetes-to-7500-nodes>
- [61] K. Ousterhout, P. Wendell, M. Zaharia, and I. Stoica, “Sparrow: Distributed, Low Latency Scheduling,” in *SOSP*, 2013.
- [62] P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warriar, N. Mahalingam, and R. Bianchini, “POLCA: Power Oversubscription in LLM Cloud Providers,” *arXiv preprint arXiv:2308.12908*, 2023.
- [63] P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warriar, N. Mahalingam, and R. Bianchini, “Characterizing Power Management Opportunities for LLMs in the Cloud,” in *ASPLOS*, 2024.
- [64] P. Patel, Z. Gong, S. Rizvi, E. Choukse, P. Misra, T. Anderson, and A. Sriraman, “Towards Improved Power Management in Cloud GPUs,” in *IEEE CAL*, 2023.
- [65] P. Patel, K. Lim, K. Jhunhunwalla, A. Martinez, M. Demoulin, J. Nelson, I. Zhang, and T. Anderson, “Hybrid Computing for Interactive Datacenter Applications,” *arXiv preprint arXiv:2304.04488*, 2023.
- [66] R. Pope, S. Douglas, A. Chowdhury, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal, and J. Dean, “Efficiently Scaling Transformer Inference,” in *MLSys*, 2023.
- [67] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, “Language Models are Unsupervised Multitask Learners,” *OpenAI blog*, 2019.
- [68] F. Romero, Q. Li, N. J. Yadwadkar, and C. Kozyrakis, “INFaaS: Automated Model-less Inference Serving,” in *USENIX ATC*, 2021.
- [69] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, J. Tow, A. M. Rush, S. Biderman, A. Webson, P. S. Ammanamanchi, T. Wang, B. Sagot, N. Muennighoff, A. V. del Moral, O. Ruwase, R. Bawden, S. Bekman, A. McMillan-Major, I. Beltagy, H. Nguyen, L. Saulnier, S. Tan, P. O. Suarez, V. Sanh, H. Laurençon, Y. Jernite, J. Launay, M. Mitchell, and C. Raffel, “BLOOM: A 176B-Parameter Open-access Multilingual Language Model,” *arXiv preprint arXiv:2211.05100*, 2022.
- [70] P. Schmid. Fine-tune FLAN-T5 XL/XXL using DeepSpeed & Hugging Face Transformers. [Online]. Available: <https://www.philschmid.de/fine-tune-flan-t5-deepspeed>
- [71] P. Schmid, O. Sansevero, P. Cuenca, and L. Tunstall. Llama 2 is here - Get it on Hugging Face. [Online]. Available: <https://huggingface.co/blog/llama2>
- [72] M. Schwarzkopf, A. Konwinski, M. Abd-El-Malek, and J. Wilkes, “Omega: Flexible, Scalable Schedulers for Large Compute Clusters,” in *EuroSys*, 2013.
- [73] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, “Fairness in Serving Large Language Models,” *arXiv preprint arXiv:2401.00588*, 2023.
- [74] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, “FlexGen: High-Throughput Generative Inference of Large Language Models with a Single GPU,” in *ICML*, 2023.
- [75] K. Shvachko, H. Kuang, S. Radia, and R. Chansler, “The Hadoop Distributed File System,” in *MSST*, 2010.
- [76] K. Van Craeynest, A. Jaleel, L. Eeckhout, P. Narvaez, and J. Emer, “Scheduling Heterogeneous Multi-cores Through Performance Impact Estimation (PIE),” *ACM SIGARCH Computer Architecture News*, 2012.
- [77] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, E. Kaiser, and I. Polosukhin, “Attention is All You Need,” in *NeurIPS*, 2017.
- [78] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. v. Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest, and A. M. Rush, “Transformers: State-of-the-art Natural Language Processing,” in *EMNLP*, 2020.
- [79] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, “Fast Distributed Inference Serving for Large Language Models,” *arXiv preprint arXiv:2305.05920*, 2023.
- [80] H. Yang, Q. Chen, M. Riaz, Z. Luan, L. Tang, and J. Mars, “PowerChief: Intelligent Power Allocation for Multi-stage Applications to Improve Responsiveness on Power Constrained CMP,” in *ISCA*, 2017.
- [81] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, “Orca: A Distributed Serving System for Transformer-Based Generative Models,” in *OSDI*, 2022.
- [82] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing,” in *NSDI*, 2012.
- [83] C. Zhang, M. Yu, W. Wang, and F. Yan, “MArk: Exploiting Cloud Services for Cost-Effective, SLO-Aware Machine Learning Inference Serving,” in *USENIX ATC*, 2019.
- [84] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. Diab, X. Li, X. V. Lin *et al.*, “OPT: Open Pre-trained Transformer Language Models,” *arXiv preprint arXiv:2205.01068*, 2022.
- [85] W. Zhu, “Analysis of JSQ Policy on Soft Real-time Scheduling in Cluster,” in *HPCAsia*, 2000.

APPENDIX

A. Abstract

We open source critical components needed to evaluate Splitwise; these could be repurposed to also evaluate future LLM inference serving systems. Our artifact includes:

- Production traces from two LLM inference services at Microsoft Azure.
- A prototype implementation of Splitwise’s KV-cache transfer mechanism in vLLM [51].
- SplitwiseSim, a discrete event simulator to evaluate model serving in LLM inference clusters.

Artifact functionality was only tested for the traces and SplitwiseSim due to limited hardware availability.

B. Artifact check-list (meta-information)

- **Data set:** Production traces available as a part of the artifact.
- **Run-time environment:** Linux / Ubuntu.
- **Hardware:** Two machines connected over GPU Infiniband for the vLLM prototype (e.g. NVIDIA DGX-A100, NVIDIA DGX-H100). x86-64 CPU machine for SplitwiseSim.
- **Publicly available?:** Yes.
- **Code licenses (if publicly available?):** MIT.
- **Data licenses (if publicly available?):** CC-BY.
- **Archived (provide DOI?):** 10.5281/zenodo.11003049.

C. Description

How to access. The entire artifact is available as an archive on Zenodo: <https://doi.org/10.5281/zenodo.11003049>. Individual components are also available online as follows:

- The production traces can be downloaded from the Azure Public Dataset GitHub repository [4].
- The KV-cache transfer prototype can be downloaded from the vLLM GitHub repository, currently available as a pull request [1].

[54] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer 和 V. Stoyanov, “RoBERTa: 一种稳健优化的 BE RT 预训练方法”, *arXiv preprint arXiv:1907.11692*, 2019. [55] Z. Liu, J. Wang, T. Dao, T. Zhou, B. Yuan, Z. Song, A. Shrivastava, C. Zhang, Y. Tian, C. Re 和 B. Chen, “似曾相识: 推理时高效大语言模型稀疏性”, *ICML*, 2023 年. [56] Meta. 隆重推出 AI Research SuperCluster — Meta 用于 AI 研究的尖端 AI 超级计算机。[在线的]。可用: <https://ai.facebook.com/blog/ai-rsc/> [57] “Azure OpenAI 服务”, Microsoft Azure, 2022 年。[在线]。可用: <https://azure.microsoft.com/en-us/products/ai-services/openai-service> [58] R. Mittal, A. Shpiner, A. Panda, E. Zahavi, A. Krishnamurthy, S. Ratnasamy 和 S. Shenker, “重新审视 RDMA 的网络支持”, *arXiv preprint arXiv:1806.08159*, 2018 年. [59] P. Moritz, R. Nishihaara, S. Wang, A. Tumanov, R. Liaw, E. Liang, M. Elibol, Z. Yang, W. Paul, M. I. Jordan *et al.*, “Ray: 新兴人工智能应用的分布式框架”, *OSDI*, 2018 年. [60] OpenAI. 将 Kubernetes 扩展到 7,500 个节点。[在线的]。可用: <https://openai.com/research/scaling-kubernetes-to-7500-nodes> [61] K. Ousterhout, P. Wendell, M. Zaharia 和 I. Stoica, “Sparrow: 分布式、低延迟调度”, *SOSP*, 2013 年. [62] P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warriar, N. Mahalingam 和 R. Bianchini, “POLCA: LLM 云提供商的超额认购”, *arXiv preprint arXiv:2308.12908*, 2023 年. [63] P. Patel, E. Choukse, C. Zhang, Í. Goiri, B. Warriar, N. Mahalingam 和 R. Bianchini, “描述云端大语言模型源管理机会”, *ASPLOS*, 2024 年. [64] P. Patel, Z. Kong, S. Rizvi, E. Choukse, P. Misra, T. Anderson 和 A. Sriraman, “迈向改进云 GPU 的电源管理”, *IEEE CAL*, 2023. [65] P. Patel, K. Lim, K. Jhunjhunwalla, A. Martinez, M. Demoulin, J. Nelson 和 I. Zhang 和 T. Anderson, “交互式数据中心应用的混合计算”, *arXiv preprint arXiv:2304.04488*, 2023. [66] R. Pope, S. Douglas, A. Chowdhery, J. Devlin, J. Bradbury, J. Heek, K. Xiao, S. Agrawal 和 J. Dean, “有效扩展 Transformer 推理”, *MLSys*, 2023 年. [67] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei 和 I. Sutskever, “语言模型是无监督的多任务学习者”, *OpenAI blog*, 2019 年. [68] F. Romero, Q. Li, N. J. Yadwadkar, 和 C. Kozyrakis, “INFaaS: 自动化无模型推理服务”, in *USENIX ATC*, 2021. [69] T. L. Scao, A. Fan, C. Akiki, E. Pavlick, S. Ilić, D. Hesslow, R. Castagné, A. S. Luccioni, F. Yvon, M. Gallé, J. Tow, A. M. Rush, S. Biderman, A. Webson, P. S. Ammanamanchi, T. Wang, B. Sagot, N. Muennighoff, A. V. del Moral, O. Ruwase, R. Bawden, S. Bekman, A. McMillan-Major, I. Beltagy, H. Nguyen, L. Saulnier, S. Tan, P. O. Suarez, V. Sanh, H. Laurençon, Y. Jernite, J. Launay, M. Mitchell 和 C. Raffel, “BLOOM: 176B 参数开放访问多语言语言模型”, *arXiv preprint arXiv:2211.05100*, 2022 年. [70] P. Schmid. 使用 DeepSpeed 和 Hugging Face Transformers 微调 FLAN-T5 XL/XXL。[在线的]。可用: <https://www.philschmid.de/fine-tune-flan-t5-deepspeed> [71] P. Schmid, O. Sans eviero, P. Cuenca 和 L. Tunstall. Llama 2 就在这里 - 在 Hugging Face 上获取它。[在线的]。可用: <https://huggingface.co/blog/llama2> [72] M. Schwartzkopf, A. Konwinski, M. Abd-El-Malek 和 J. Wilkes, “Omega: 用于大型计算集群的灵活、可扩展的调度程序”, *EuroSys*, 2013 年. [73] Y. Shen, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez 和 I. Stoica, “服务大型语言模型的公平性”, *arXiv preprint arXiv:2401.00588*, 2023. [74] Y. Shen, L. Cheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica 和 C. Zhu, “FlexGen: 使用单个对象对大型语言模型进行高通量生成推理” GPU”, 载于 *ICML*, 2023 年. [75] K. Shvachko, H. Kuang, S. Radia 和 R. Chansler, “Hadoop 分布式文件系统”, 载于 *MSST*, 2010 年. [76] K. Van Craeynest, A. Jaleel, L. Eeckhout, P. Narvaez 和 J. Emer, “通过调度异构多核”性能影响评估 (PIE)”, *ACM SIGARCH Computer Architecture News*, 2012 年. [77] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, 和 I. Polosukhin, “注意力就是你所需要的”, *NeurIPS*, 2017 年.

[78] T. Wolf, L. Debut, V. Sanh, J. Chaumond, C. Delangue, A. Moi, P. Cistac, T. Rault, R. Louf, M. Funtowicz, J. Davison, S. Shleifer, P. v. Platen, C. Ma, Y. Jernite, J. Plu, C. Xu, T. L. Scao, S. Gugger, M. Drame, Q. Lhoest 和 A. M. Rush, “变形金刚: 最先进的自然语言处理”, *EMNLP*, 2020 年. [79] B. Wu, Y. zhong, Z. 张, G. Huang, X. Liu 和 X. Jin, “大型语言模型的快速分布式推理服务”, *arXiv preprint arXiv:2305.05920*, 2023 年. [80] H. Yang, Q. Chen, M. Riaz, Z. Luan, L. Tang 和 J. Mars, “PowerChief: 多级应用的智能功率分配, 以提高功率受限 CMP 的响应能力”, *ISCA*, 2017 年. [81] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim 和 B.-G. Chun, “Orca: 基于 Transformer 的生成模型的分布式服务系统”, *OSDI*, 2022 年. [82] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauly, M. J. Franklin, S. Shenker 和 I. Stoica, “弹性分布式数据集: 容错抽象”内存集群计算”, *NSDI*, 2012. [83] C. Zhang, M. Yu, W. Wang 和 F. Yan, “MArk: 利用云服务实现经济高效、SLO 感知的机器学习推理服务”, *USENIX ATC*, 2019. [84] S. Zhang, S. Roller, N. Goyal, M. Artetxe, M. Chen, S. Chen, C. Dewan, M. D. Iab, X. Li, X. V. Lin *et al.*, “OPT: 开放式预训练 Transformer 语言模型”, *arXiv preprint arXiv:2205.01068*, 2022 年. [85] W. Zhu, “JSQ 集群软实时调度策略分析”, *HPCAsia*, 2000 年.

附录

A. Abstract

我们开源评估 Splitwise 所需的关键组件; 这些可以重新用于评估未来的大语言模型服务系统。我们的工件包括:

- 来自 Microsoft Azure 的两个 LLM 推理服务的生产跟踪。
- vLLM 中 Splitwise KV 缓存传输机制的原型实现 [51]。
- SplitwiseSim, 一个离散事件模拟器, 用于评估 LLM 推理集群中的模型服务。

由于硬件可用性有限, 仅针对迹线和 SplitwiseSim 测试了 Artifact 功能。

B. Artifact check-list (meta-information)

- 数据集: 作为工件的一部分提供的生产跟踪。
- 运行环境: Linux / Ubuntu。
- 硬件: 通过 GPU Infiniband 连接的两台机器用于 vLLM 原型 (e.g. NVIDIA DGX-A100, NVIDIA DGX-H100)。
- SplitwiseSim 的 x86-64 CPU 机器。
- 公开吗?: 是的。
- 代码许可证 (如果公开可用)? : 麻省理工学院。
- 数据许可证 (如果公开可用)? : CC-BY。
- 已存档 (提供 DOI)? : 10.5281/zenodo.11003049。

C. Description

如何访问。整个工件可作为 Zenodo 上的存档: <https://doi.org/10.5281/zenodo.11003049>。各个组件也可以在线获取, 如下所示:

- 生产跟踪可以从 Azure 公共数据集 GitHub 存储库 [4] 下载。
- KV 缓存传输原型可以从 vLLM GitHub 存储库下载, 目前可作为拉取请求 [1]。

- SplitwiseSim, and the associated experiment and plotting scripts, can be downloaded from a separate GitHub repository [20].

Hardware dependencies. The KV-cache transfer prototype requires two GPU machines connected over Infiniband, such as NVIDIA DGX-A100s or NVIDIA DGX-H100s. SplitwiseSim requires a standard x86-64 CPU machine; multiple machines may be used to parallelize simulation runs.

Software dependencies. The KV-cache transfer prototype is built on top of vLLM [51] and MSCCL++ [11]. SplitwiseSim depends on a small set of publicly available Python packages, which can be installed via the included `requirements.txt`.

Data sets. Coding and conversation traces from Microsoft Azure are available online as a part of the artifact release [4].

D. Installation and Experiment Workflow

Please refer to the README files within the artifact for installation and usage instructions.

E. Methodology

Submission, reviewing and badging methodology:

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <http://cTuning.org/ae/submission-20201122.html>
- <http://cTuning.org/ae/reviewing-20201122.html>

- SplitwiseSim 以及相关的实验和绘图脚本可以从单独的 GitHub 存储库下载 [20]。

硬件依赖性。KV 缓存传输原型需要两台通过 Infiniband 连接的 GPU 机器，例如 NVIDIA DGX-A100 或 NVIDIA DGX-H100。SplitwiseSim 需要标准 x86-64 CPU 机器；可以使用多台机器来并行模拟运行。

软件依赖性。KV 缓存传输原型构建在 vLLM [51] 和 MSC CL++ [11] 之上。SplitwiseSim 依赖于一小组公开可用的 Python 包，可以通过包含的 requests.txt 安装。

数据集。来自 Microsoft Azure 的编码和对话跟踪可作为工件版本的一部分在线获取 [4]。

D. Installation and Experiment Workflow

请参阅工件内的自述文件以获取安装和使用说明。

E. Methodology

提交、审核和标记方法：

- <https://www.acm.org/publications/policies/artifact-review-and-badging-current>
- <http://cTuning.org/ae/submission-20201122.html>
- <http://cTuning.org/ae/reviewing-20201122.html>